A

Project Report

on

ROAD TRAFFIC ANALYSIS USING YOLOv4 AND DEEP SORT

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE)

Submitted by

Sandaka Venkata Rajesh	22A31A4354
Atapakala Devi Harshitha	22A31A4303
Leela Prasanna Yeleti	22A31A4310
Sangadi Maharaju	23A35A4309

Under the esteemed supervision of

Mrs. P. Seshu Kumari, M.Tech,

Assistant Professor in CSE (AI)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

PRAGATI ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, KAKINADA, Accredited by NBA & NAAC with 'A+' grade)

ADB Road, Surampalem, Near Peddapuram, Kakinada District, AP-533437

2025 - 2026

ROAD TRAFFIC ANALYSIS USING YOLOv4 AND DEEP SORT

A

Project Report

on

ROAD TRAFFIC ANALYSIS USING YOLOv4 AND DEEP SORT

Submitted in partial fulfillment of the requirement for the

award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTERSCIENCEAND ENGINEERING

(ARTIFICIAL INTELLIGENCE)

Submitted by

Sandaka Venkata Rajesh	22A31A4354
Atapakala Devi Harshitha	22A31A4303
Leela Prasanna Yeleti	22A31A4310
Sangadi Maharaju	23A35A4309

Under the esteemed supervision of

Mrs. P. Seshu Kumari, M.Tech,

Assistant Professor in CSE (AI)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

PRAGATI ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, KAKINADA, Accredited by NBA & NAAC with 'A+' grade)

ADB Road, Surampalem, Near Peddapuram, Kakinada District, AP-533437

2025 - 2026

PRAGATI ENGINEERINGCOLLEGE

(AUTONOMOUS)

(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada, Accredited by NBA & NAAC with 'A+' grade)

ADB Road, Surampalem, Near Peddapuram, Kakinada District, AP-533437

CERTIFICATE

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

This is to certify that the project report entitled "**ROAD TRAFFIC ANALYSIS USING YOLOv4 AND DEEP SORT**" is being submitted by **Sandaka Venkata Rajesh** (22A31A4354)**, Atapakala Devi Harshitha (**22A31A4303**), Leela Prasanna Yeleti (**22A31A4310**), Sangadi Maharaju (**23A35A4309**),** in partial fulfilment for the award of the Degree of **Bachelor of Technology**, during the year **2025-2026** in CSE (ARTIFCIAL INTELLIGENCE) of Pragati Engineering College, for the record of a bonafide work carried out by them.

Project Supervisor:

Mrs. P. Seshu Kumari, M.Tech.,

Assistant Professor

Department of CSE (AI)

Pragati Engineering College (A)

Head of the Department:

Mrs. K. Lakshmi Viveka, M.Tech., (Ph.D),

Associate Professor & HOD

Department of CSE (AI)

Pragati Engineering College (A)

External Signature

ACKNOWLEDGEMENT

We express our thanks to project supervisors **Mrs. P. SESHU KUMARI, Assistant Professor of CSE (ARTIFICIAL INTELLIGENCE),** who deserves a special note of thanks and gratitude, for having extended their fullest co- operation and guidance, without this, project would never have materialized.

We wish to express our special thanks to Mrs. **K. LAKSHMI VIVEKA, Associate Professor & HOD of CSE (ARTIFICIAL INTELLIGENCE),** for giving us guidelines & encouragement.

We wish to express our special thanks to our beloved **Dr. K. SATYANARAYANA, Professor & Director (Academics)** for giving guidelines and encouragement.

We wish to express our special thanks to our beloved **Dr. G. NARESH, Professor & Principal** for giving guidelines and encouragement.

We wish to express sincere gratitude to our beloved and respected **Sri. M. SATISH, Vice- President**, **Sri. M. V. HARANATHA BABU, Director (Management)** and **Dr. P. KRISHNA RAO, Chairman** for their encouragement and blessings.

We are thankful to all our faculty members of the Department for their valuable suggestions. Our sincere thanks are also extended to all the teaching and non-teaching staff of Pragati Engineering College.

SANDAKA VENKATA RAJESH 22A31A4354

ATAPAKALA DEVI HARSHITHA 22A31A4303

LEELA PRASANNA YELETI 22A31A4310

SANGADI MAHARAJU 23A35A4309

ABSTRACT

Road Traffic Analysis Using YOLO-V4 & Deep Sort" is an AI-powered system designed to enhance traditional traffic monitoring by providing real-time vehicle detection and tracking. With the rapid increase in urban populations and vehicular density, existing traffic management systems struggle with inefficiencies, scalability issues, and delayed processing. This project addresses these challenges by integrating deep learning techniques, specifically YOLO-V4 for object detection and Deep Sort for multi-object tracking, to analyze live and recorded video streams.

The system detects vehicles, assigns unique IDs, and tracks them across multiple frames, offering insights into congestion levels, movement trends, and road conditions. Advanced preprocessing techniques, such as non-max suppression, bounding box filtering, and Kalman filtering, improve accuracy and reduce detection inconsistencies. Implemented using Python, TensorFlow, OpenCV, and NumPy, the system ensures high performance and adaptability across diverse environments, including urban roads, highways, and smart city infrastructures.

By automating traffic surveillance, this project assists transportation authorities in optimizing road usage, minimizing congestion, and enhancing public safety. Future advancements include predictive analytics for traffic flow estimation, IoT-based adaptive traffic control, and cloud integration for large-scale deployment. The system's potential applications extend to smart toll collection, accident detection, and autonomous vehicle navigation, making it a significant contribution to the evolution of intelligent transportation systems.

Recent advancements in artificial intelligence (AI) and deep learning offer new possibilities for real-time traffic monitoring and analysis. By leveraging AI-driven solutions, cities can implement intelligent traffic management systems that optimize road usage, reduce congestion, and improve public safety. Deep Sort, a multi-object tracking algorithm, assigns unique IDs to detected vehicles, enabling consistent tracking across frames. The system enables authorities to monitor congestion, detect traffic violations, and optimize infrastructure planning.

Future research could explore autonomous traffic control mechanisms and integration with smart city frameworks, offering a scalable solution for next-generation urban traffic system. By eliminating the inefficiencies of manual monitoring, this solution enables data-driven decision-making, helping cities manage their transportation networks more effectively. This project integrates these insights, utilizing AI-powered analytics for predictive traffic flow optimization, making it adaptable for diverse urban infrastructures.

LIST OF CONTENTS

LIST OF FigS

LIST OF TABLES

CHAPTER – 1
INTRODUCTION

Road traffic congestion is a growing problem in urban areas worldwide, causing delays, increasing fuel consumption, and contributing to air pollution. Traditional traffic management systems, which rely on manual surveillance, static sensors, and predefined signal timings, struggle to adapt to fluctuating traffic patterns. These conventional methods lack the ability to process and analyze real-time traffic data efficiently, making them ineffective in addressing the complex challenges of modern transportation networks. Recent advancements in artificial intelligence (AI) and deep learning offer new possibilities for real-time traffic monitoring and analysis. By leveraging AI-driven solutions, cities can implement intelligent traffic management systems that optimize road usage, reduce congestion, and improve public safety.

The integration of deep learning models like YOLO-V4 and Deep Sort in traffic analysis provides a scalable and automated approach to vehicle detection and tracking. YOLO-V4, a state-of-the-art object detection model, offers fast and accurate identification of vehicles in real-time video feeds. Deep Sort, a multi-object tracking algorithm, assigns unique IDs to detected vehicles, enabling consistent tracking across frames. By combining these technologies, the proposed system delivers precise traffic insights, including vehicle density, movement trajectories, and congestion hotspots. Such a system not only benefits traffic authorities in decision-making but also paves the way for smart city applications, including adaptive traffic light control and autonomous vehicle coordination.

This project aims to develop an AI-powered traffic monitoring solution that provides real- time data on vehicle movements, congestion levels, and road conditions. The system is designed to handle live video streams from traffic cameras, process the data using deep learning models, and generate actionable insights for traffic management authorities. By eliminating the inefficiencies of manual monitoring, this solution enables data-driven decision-making, helping cities manage their transportation networks more effectively. Future enhancements to the system may include integration with IoT sensors for additional data collection, predictive analytics for congestion forecasting, and cloud-based deployment for large-scale traffic monitoring. With the growing demand for intelligent transportation systems, this project represents a significant step toward the automation and optimization of road traffic analysis.

1.1 Purpose

The primary purpose of Road Traffic Analysis Using YOLO-V4 & Deep Sort is to develop an AI-driven traffic monitoring system for real-time vehicle detection, tracking, and congestion analysis. With increasing urbanization and vehicle density, traditional traffic management methods—such as manual surveillance and static sensors—are inefficient, leading to delays, human errors, and scalability issues. This project integrates YOLO-V4 for object detection and Deep Sort for multi-object tracking, ensuring accurate traffic flow analysis and automated insights to help authorities identify congestion hotspots, predict peak traffic periods, and optimize road infrastructure planning.

Supporting smart city initiatives, the system enhances adaptive traffic control, public transportation efficiency, and road safety. AI-driven analytics help reduce fuel consumption, minimize travel delays, and improve urban mobility while promoting sustainable traffic solutions. Designed for scalability and adaptability, the system can be deployed across highways, urban roads, intersections, and toll booths. Future enhancements may include IoT-enabled sensors, predictive analytics for accident prevention, and cloud-based deployment for large-scale traffic management.

By automating traffic surveillance, optimizing traffic flow, and improving decision-making, this project aims to revolutionize urban traffic management, reduce congestion, and create a smarter, more efficient transportation network. Another important purpose of this project is to assist traffic management authorities in identifying congestion hotspots and understanding traffic behavior. By analyzing vehicle density and movement patterns, the system can help authorities detect areas where traffic congestion frequently occurs. This information can be used to improve traffic signal timing, optimize road infrastructure, and reduce traffic delays.

The system also contributes to improving road safety by providing detailed insights into traffic flow and vehicle movement. Monitoring vehicle behavior can help detect unusual traffic patterns, potential accidents, or rule violations. This information can assist traffic authorities in taking preventive measures and improving road safety. The project is designed to be scalable and adaptable, allowing it to be deployed in different traffic environments such as highways, urban roads, intersections, and toll plazas. This increase has resulted in severe traffic congestion, longer travel times, higher fuel consumption, and increased environmental pollution. Efficient traffic monitoring and management have therefore become essential for maintaining smooth transportation systems and ensuring road safety.

1.2 Scope

The Scope of Road Traffic Analysis Using YOLO-V4 & Deep Sort includes real-time detection and tracking of vehicles by analyzing both live and recorded video feeds, providing data-driven solutions for congestion management. Designed for deployment on urban roads, highways, toll booths, and high-traffic intersections, the system adapts to various traffic conditions, ensuring flexibility and scalability. Future enhancements may include IoT-enabled sensors for improved monitoring, predictive analytics for accident prevention, and cloud-based deployment for large-scale traffic data processing.

By automating traffic surveillance and enhancing decision-making, Road Traffic Analysis Using YOLO-V4 & Deep Sort contributes to sustainable and intelligent transportation solutions, improving urban mobility, reducing congestion, and optimizing traffic flow. Additionally, the system can assist law enforcement agencies in monitoring traffic violations, enforcing regulations, and enhancing overall road safety.

One of the main aspects of the project's scope is real-time vehicle detection using deep learning techniques. The system analyzes video input from traffic cameras and identifies vehicles present in each frame using the YOLO-V4 object detection algorithm. This allows the system to recognize different types of vehicles, including cars, buses, trucks, and motorcycles.

Another important aspect of the project is vehicle tracking. The Deep SORT algorithm enables the system to track individual vehicles across multiple video frames. Each vehicle is assigned a unique tracking ID, which allows the system to follow its movement and avoid counting the same vehicle multiple times. This tracking capability helps generate accurate traffic statistics and vehicle counts.

The system also focuses on analyzing traffic density and congestion levels. By monitoring the number of vehicles passing through a specific area, the system can estimate traffic volume and identify periods of high congestion. This information can help traffic authorities manage road networks more effectively.

Manual monitoring requires significant human effort and is prone to errors and delays. Sensor-based systems are expensive to install and maintain, and they often fail to provide detailed information about vehicle movement and traffic patterns. Additionally, conventional systems cannot efficiently process large volumes of real-time traffic data, which limits their ability to support intelligent traffic management.

1.3 Motivation

The motivation behind "Road Traffic Analysis Using YOLO-V4 & Deep Sort" stems from the growing challenges in urban traffic management due to rapid urbanization and increasing vehicle density. Traditional traffic monitoring methods, such as manual surveillance and sensor-based systems, are inefficient, error-prone, and lack scalability, leading to congestion and accidents. The need for an intelligent, automated system that provides real-time traffic insights and optimizes traffic flow has never been more critical.

By integrating YOLO-V4 for object detection and Deep Sort for multi-object tracking, this project offers a cutting-edge AI- driven solution for real-time traffic analysis, improving decision-making for traffic authorities. The system's ability to predict congestion hotspots and support adaptive traffic control aligns with smart city initiatives. Additionally, AI-powered traffic monitoring can reduce fuel consumption, minimize emissions, and contribute to a sustainable urban mobility ecosystem.

With its real-time analytics, high accuracy, and adaptability, this project aims to revolutionize traditional traffic surveillance and create safer, more intelligent road networks. Another motivating factor for this project is the growing need for intelligent transportation systems as part of smart city initiatives. Smart cities require advanced technologies to manage transportation networks efficiently and ensure sustainable urban development. AI-based traffic monitoring systems can provide valuable insights into traffic behaviour, enabling authorities to implement adaptive traffic control strategies.

The project is also motivated by the need to improve road safety. Traffic congestion and poor traffic management often lead to accidents and delays. By analyzing traffic patterns and identifying congestion hotspots, the system can help authorities implement preventive measures and improve road safety conditions.

Furthermore, the use of deep learning models such as YOLO-V4 and Deep SORT demonstrates the potential of modern AI technologies in solving real-world problems. The project highlights how these advanced algorithms can be applied to practical applications such as traffic monitoring and urban planning.

Deep learning models are capable of analyzing visual data from cameras and automatically identifying objects such as vehicles, pedestrians, and road signs. These technologies allow traffic systems to detect and track vehicles in real time, analyze traffic flow patterns, and provide valuable insights for traffic management authorities.

1.4 Problem Statement

Rapid urbanization and the rise in vehicle numbers have placed immense pressure on existing traffic management systems, leading to severe congestion and increased road accidents. “Road Traffic Analysis Using YOLO-V4 & Deep Sort” addresses these inefficiencies by providing an AI-powered solution for real-time vehicle detection, tracking, and congestion analysis. Traditional methods, such as manual monitoring and static sensors, are unreliable, labor-intensive, and unable to handle high traffic volumes effectively.

With the rise in vehicle density, there is a critical need for an automated, data-driven approach to traffic monitoring. This project leverages YOLO-V4 for high-speed object detection and Deep Sort for accurate multi-object tracking, enabling authorities to identify congestion patterns, monitor traffic violations, and optimize road infrastructure.

The system provides continuous, real-time analysis of traffic conditions, allowing for better traffic regulation and improved public transportation efficiency. Designed for deployment in highways, intersections, and tollbooths, the system ensures adaptability across diverse environments. By offering an intelligent, automated solution, this project aims to improve transportation efficiency, minimize congestion, and enhance overall urban mobility.

Another major problem is the lack of intelligent traffic analysis tools that can automatically detect and track vehicles in real time. Without automated analysis, traffic authorities find it difficult to identify congestion hotspots, monitor vehicle flow, and make informed decisions regarding traffic management. With the rapid increase in urban populations and vehicular density, existing traffic management systems struggle with inefficiencies, scalability issues, and delayed processing.

To address these challenges, this project proposes the development of an AI-based traffic monitoring system that uses deep learning algorithms to detect and track vehicles in real time. By leveraging YOLO-V4 for object detection and Deep SORT for vehicle tracking, the system aims to provide accurate traffic analysis and automated monitoring capabilities.

The proposed solution aims to reduce dependency on manual monitoring, improve the accuracy of traffic analysis, and provide valuable insights for traffic management authorities. By implementing an intelligent traffic analysis system, cities can improve traffic flow, reduce congestion, enhance road safety, and support the development of modern intelligent transportation system.

CHAPTER – 2
LITERATURE SURVEY

The evolution of traffic monitoring systems has progressed from traditional manual methods to AI-driven intelligent systems. Earlier approaches relied on inductive loop detectors, CCTV cameras, and manual traffic counting, which were labor-intensive and lacked real-time adaptability. With the rise in vehicle density, such methods became inefficient in handling urban traffic congestion and road safety concerns. Recent advancements in computer vision and deep learning have enabled automated vehicle detection, tracking, and congestion analysis. Research on object detection models like YOLO (You Only Look Once) highlights its ability to process high-speed traffic data with enhanced accuracy. Similarly, tracking algorithms such as Deep Sort (Simple Online and Realtime Tracker) provide robust multi- object tracking, ensuring continuous monitoring of vehicles in dynamic environments. These AI-powered technologies form the foundation of this project, aligning with trends in intelligent transportation systems (ITS) for better urban mobility.

Traditional traffic management tools, including radar-based sensors and rule-based tracking algorithms, have demonstrated certain efficiencies but are constrained by environmental conditions, occlusions, and real-time processing limitations. Studies on Kalman filtering and Hungarian algorithms suggest improvements in object tracking; however, these models often struggle with large-scale urban traffic due to frequent object occlusions and overlapping vehicles. Comparative studies on deep learning models for traffic surveillance emphasize the superiority of AI-based object detection over conventional vision-based techniques. The combination of YOLO-V4 and Deep Sort in this project enhances accuracy in vehicle classification, congestion mapping, and traffic violation detection.

The literature further discusses challenges such as high computational costs, real-time data processing constraints, and scalability for large-scale deployment. Research on cloud-based traffic monitoring solutions highlights the advantages of leveraging IoT-enabled smart sensors and big data analytics to optimize congestion management. These conventional methods lack the ability to process and analyze real-time traffic data efficiently, making them ineffective in addressing the complex challenges of modern transportation networks. This project integrates these insights, utilizing AI-powered analytics for predictive traffic flow optimization, making it adaptable for diverse urban infrastructures. Future research could explore autonomous traffic control mechanisms and integration with smart city frameworks, offering a scalable solution for next-generation urban traffic system.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 Introduction

The "Road Traffic Analysis Using YOLO-V4 & Deep Sort" project presents an AI-driven solution to enhance traffic monitoring and management. With rapid urbanization and increasing vehicle density, traditional surveillance systems struggle with inefficiencies, errors, and limited scalability. This project integrates YOLO-V4 for real-time vehicle detection and Deep Sort for accurate multi-object tracking, ensuring a robust and scalable approach to traffic analysis. The system enables authorities to monitor congestion, detect traffic violations, and optimize infrastructure planning. By leveraging deep learning models, the project provides high-precision analytics, offering real-time insights for better traffic regulation and road safety.

This section examines the system's architecture, compares it with existing traffic monitoring methods, and highlights its key advantages. The study references the integration of AI-based models with traffic surveillance datasets, forming a structured approach to intelligent traffic monitoring. The subsequent subsections discuss the limitations of current systems, the advantages of the proposed approach, and a feasibility study covering economic, operational, and technical dimensions.

Another important aspect of the system analysis is evaluating the feasibility and efficiency of the proposed system. The proposed AI-based system aims to provide a scalable and cost-effective solution that can integrate with existing surveillance infrastructure such as CCTV cameras. This makes the system practical for deployment in real-world traffic environments such as highways, intersections, toll plazas, and urban road networks.

In addition to improving traffic monitoring efficiency, the proposed system also supports the development of intelligent transportation systems and smart city initiatives. By using artificial intelligence and deep learning techniques, the system enables automated and data-driven traffic management, which can help reduce traffic congestion, improve road safety, and enhance overall urban mobility. It helps cities implement smarter traffic management strategies and improve urban mobility.

3.2 Existing System

Current traffic monitoring systems rely on manual surveillance, inductive loop detectors, and fixed CCTV cameras. Manual methods require human intervention, leading to inefficiencies and delays in detecting traffic congestion or violations. Inductive loop detectors are embedded in roads but suffer from maintenance issues and limited scalability. Traditional CCTV cameras provide video footage, but without AI-based analytics, they require manual monitoring, which is time-consuming and prone to errors. Additionally, rule-based tracking algorithms used in conventional systems lack adaptability in dynamic environments. They struggle with occlusions, poor lighting conditions, and high-density traffic scenarios. These limitations hinder their effectiveness in handling real-time traffic analysis and decision- making.

3.2.1 Drawbacks

Existing traffic monitoring systems have several drawbacks that impact their efficiency and accuracy. Manual surveillance is labor-intensive, requiring continuous monitoring by traffic personnel, leading to human errors and inconsistencies. Inductive loop detectors, though automated, are expensive to install and maintain, limiting their deployment in large-scale urban areas. Moreover, these detectors cannot differentiate between vehicle types or detect real-time violations. CCTV-based monitoring systems depend on human operators, making real-time traffic assessment challenging. They lack automated congestion detection and predictive analytics, which are essential for modern traffic management. Additionally, traditional rule-based tracking fails in complex scenarios where vehicles overlap, occlude each other, or move unpredictably.

The lack of intelligent decision-making and automation reduces the effectiveness of these systems in handling high-traffic volumes. Furthermore, these legacy systems often struggle with poor visibility conditions, such as fog, rain, or nighttime, making accurate vehicle detection and tracking even more difficult. Another drawback is the high cost associated with installing and maintaining sensor-based systems such as inductive loop detectors. These sensors require physical installation on roads, which can be expensive and disruptive to traffic during installation and maintenance.Traditional traffic monitoring systems also lack automation. They are not capable of automatically detecting vehicles or tracking their movement across video frames. This makes it difficult to analyze traffic patterns or estimate traffic density accurately.

3.3 Proposed System

The proposed system leverages deep learning models—YOLO-V4 for vehicle detection and Deep Sort for object tracking—to create an automated, AI-powered traffic monitoring solution. The system processes real-time traffic footage, accurately identifying and tracking multiple vehicles in diverse environmental conditions. It integrates predictive analytics to anticipate congestion hotspots and provides data-driven insights for traffic management authorities. This approach eliminates the inefficiencies of manual monitoring by offering automated, high-accuracy detection and tracking. The AI models are optimized for speed and scalability, ensuring real-time performance even in high-density traffic conditions. The system's adaptability allows deployment at intersections, highways, and toll booths, making it a comprehensive solution for urban traffic management. Additionally, the system incorporates violation detection mechanisms, identifying instances of traffic rule breaches such as red-light violations and illegal lane changes. Its seamless integration with cloud-based platforms enables remote monitoring and data accessibility, enhancing decision-making for traffic authorities.

3.3.1 Advantages

The AI-powered traffic analysis system provides numerous advantages over traditional methods. Firstly, real-time vehicle detection and tracking improve the accuracy and efficiency of traffic monitoring, reducing reliance on manual intervention. The system can operate in various conditions, including low-light and high-traffic scenarios, ensuring reliability. Another key benefit is congestion prediction, allowing authorities to take proactive measures to manage traffic flow effectively. The integration of AI enhances decision-making by providing data-driven insights on road usage, accident-prone areas, and traffic rule violations. Additionally, scalability ensures deployment across multiple locations without significant infrastructure modifications. Another advantage is real-time traffic analysis. The system can analyze traffic videos in real time and provide immediate insights into traffic conditions. This allows authorities to respond quickly to congestion or accidents. The use of deep learning models also reduces maintenance costs compared to inductive loop detectors, making the system a cost-effective solution for modern traffic management.

3.4 Feasibility Study

A feasibility study assesses the viability of implementing this AI-based traffic monitoring system across economic, operational, and technical aspects. This evaluation ensures that the system is practical, cost-effective, and capable of seamless integration into existing traffic infrastructure.

3.4.1 Economic Feasibility

The economic feasibility of the proposed AI-driven traffic monitoring system is supported by cost-effective deployment and long-term operational savings. Traditional traffic monitoring methods, such as manual surveillance and inductive loop detectors, involve high installation, maintenance, and labor costs. In contrast, this system leverages AI models that operate on existing CCTV infrastructure, significantly reducing hardware expenses. Cloud-based processing and edge computing minimize the need for extensive on-premise hardware, ensuring scalability without major financial investments. Additionally, automation reduces the reliance on human intervention, lowering operational costs over time. While initial AI model training may require computational resources, advancements in GPU technology and cloud services optimize cost efficiency. The use of open-source framework like TensorFlow further eliminates licensing fees, making the system economically viable.

Compared to conventional monitoring tools requiring continuous maintenance, this solution offers a cost- effective alternative with high accuracy and efficientMost of the software tools used in the system, such as Python, TensorFlow, and OpenCV, are open-source and freely available. This significantly reduces software development costs.

Although the system may require computational resources such as GPUs for efficient deep learning processing, the overall cost remains lower compared to traditional sensor-based systems that require expensive road infrastructure. Economic feasibility examines whether the proposed system is financially viable. The proposed traffic monitoring system utilizes existing surveillance infrastructure such as CCTV cameras, which reduces the need for additional hardware installation.

3.4.2 Operational Feasibility

Operational feasibility evaluates the system's practicality in real-world applications. The AI- driven approach ensures automated, real-time traffic monitoring, reducing the workload on human operators. The system's ability to detect violations, track congestion, and analyze traffic patterns enhances road safety and traffic efficiency. Deployment in various urban scenarios—such as highways, intersections, and toll booths—demonstrates adaptability across different environments.

The system integrates seamlessly with existing traffic cameras and monitoring infrastructure, requiring minimal modifications for implementation. User- friendly dashboards and reporting tools further improve usability for traffic management authorities. Operational feasibility evaluates whether the system can be used effectively in real-world traffic environments.

The proposed system is designed with a user-friendly interface that allows users to upload traffic videos and analyze traffic conditions easily. Traffic monitoring authorities can use the system without requiring advanced technical knowledge. The system can also integrate with existing traffic surveillance infrastructure, making it practical for deployment in real-world scenarios. Thus, the proposed system is operationally feasible.

3.4.3 Technical Feasibility

Technical feasibility examines the technological requirements for deploying the proposed system. The deep learning models—YOLO-V4 for detection and Deep Sort for tracking—are optimized for real-time processing, ensuring minimal latency and high accuracy. The system requires GPU acceleration for efficient computation, making it compatible with modern AI framework such as TensorFlow.

The AI-powered monitoring system is designed for scalability, capable of handling high-traffic volumes without performance degradation. Cloud-based processing and IoT-enabled sensors can further enhance the system's capabilities, providing a robust infrastructure for intelligent traffic management. The seamless integration of AI models with existing surveillance networks ensures technical compatibility, making deployment feasible across various urban locations.

3.4.4 Legal Feasibility

Legal feasibility ensures that the proposed system complies with applicable laws and regulations. The system analyzes traffic videos captured by surveillance cameras and does not collect personal or sensitive data from individuals. Therefore, it does not violate privacy regulations when used for traffic monitoring purposes.

As long as the system is used responsibly for traffic analysis and road safety improvements, it complies with legal requirements. The proposed system uses widely available technologies such as Python, TensorFlow, OpenCV, and deep learning models like YOLO-V4 and Deep SORT. Some traffic monitoring systems also use radar sensors or infrared sensors to detect vehicle movement. However, these systems often struggle in complex traffic environments where multiple vehicles overlap or where environmental conditions such as rain, fog, or poor lighting affect sensor performance.

These technologies are well-supported and widely used in computer vision applications. Modern computers equipped with GPUs can efficiently process video frames and run deep learning models in real time. Moreover, existing systems often lack the ability to automatically track vehicles across frames or analyze traffic patterns in real time.

3.4.5 Social Feasibility

Social feasibility evaluates the impact of the proposed system on society. The system contributes to improving road safety and reducing traffic congestion, which benefits both commuters and transportation authorities. It helps cities implement smarter traffic management strategies and improve urban mobility. Traditional traffic monitoring systems also lack automation.

They are not capable of automatically detecting vehicles or tracking their movement across video frames. This makes it difficult to analyze traffic patterns or estimate traffic density accurately. Environmental factors also affect the performance of existing systems. Poor lighting conditions, rain, fog, and camera angle limitations can reduce the accuracy of vehicle detection and monitoring. By reducing traffic congestion and travel delays, the system also contributes to reducing fuel consumption and environmental pollution.

CHAPTER - 4

SYSTEM REQUIREMENTS AND SPECIFICATIONS

4.1 Software Requirements

The Road Traffic Analysis System is implemented using Python as the primary programming language due to its extensive support for computer vision and deep learning libraries. The system utilizes advanced object detection and tracking techniques to monitor and analyze real-time traffic conditions from video streams.

The core of the system is based on the YOLO-V4 (You Only Look Once Version 4) algorithm, which is used for real-time vehicle detection. YOLO-V4 is highly efficient and capable of detecting multiple objects such as cars, buses, trucks, and motorcycles in a single frame with high accuracy and speed.

For tracking detected vehicles across consecutive frames, the system employs the Deep SORT (Simple Online and Realtime Tracking) algorithm. Deep SORT assigns unique IDs to each detected vehicle and ensures accurate tracking even in crowded traffic scenarios. This helps in counting vehicles and analyzing movement patterns without duplication.

To handle image and video processing, libraries such as OpenCV and NumPy are used. OpenCV is responsible for capturing video input, frame extraction, and visualization, while NumPy supports numerical computations and data handling.

Operating System Compatibility

- Windows (10 and above)
- Linux (Ubuntu, Debian, CentOS)
- macOS (for development environments)

Backend Technologies

- Programming Language : Python
- Deep Learning Framework: PyTorch / TensorFlow
- Video Processing Libraries: OpenCV, NumPy
- Object Detection Model: YOLO–V4
- Tracking Algorithm: Deep SORT

Frontend Technologies (If Web Interface is Used)

- Markup & Styling: HTML, CSS
- Client-side Interaction: JavaScript
- Web Interface: Flask

Data Handling / Storage

- Input Data: Traffic video datasets used for vehicle detection and tracking during training and testing.
- Temporary Storage: Video frames and intermediate processing data are temporarily stored during real-time analysis.
- Model Files: Pre-trained YOLO-V4 weights and Deep SORT model files are stored for vehicle detection and tracking during inference.

AI & Deep Learning Stack (Production Optimized)

Table 4.1.1 – Production AI Libraries

Library	Version	Purpose
PyTorch	1.8.1	Framework for training and running
TorchVision	0.9.1	Provides video transformation utilities
CUDA (optional)	11.8 or 12.1	Enables GPU acceleration for faster YOLO-V4 detection and Deep SORT tracking computations.

Data Processing & Analytics Libraries

Table 4.1.2 – High-Performance Analytics

Library	Version	Purpose
NumPy	2.1.1	Used for numerical computations and handling arrays during video frame processing
Pandas	2.2.5	Used for traffic data handling and time-series analysis
OpenCV	4.10.0	Used for video preprocessing, frame extraction, and computer vision tasks
Matplotlib	3.8.4	Visualization of traffic analysis results

Deployment Infrastructure:

- Edge: NVIDIA Jetson AGX Orin (48 TOPS)
- Cloud: AWS EC2 G5.4xlarge (A10G GPUs)
- Orchestration: Kubernetes 1.31 + Helm 3.15

4.2 Hardware Requirements

Hardware requirements describe the computing resources necessary to develop, train, and run the proposed road traffic analysis system. Since the system relies on deep learning techniques for real-time vehicle detection and tracking, adequate processing power and memory are required to ensure efficient model execution and video processing. The hardware configuration may vary depending on whether the system is used for development, training, or deployment.The system is primarily designed to run on standard computing environments with optional GPU acceleration to improve detection speed and performance. Training the YOLO-V4 model requires moderate to high computational power, while real-time inference and vehicle tracking can run on standard desktop systems with good processing capability.

Development System Requirements

The development environment is used to implement the road traffic analysis system, train detection models, and test the application. Developers can use a standard workstation equipped with sufficient memory and storage to handle video processing, model training, and system integration.

Typical development hardware includes:

- Processor: Intel Core i5 / i7 or AMD Ryzen 5 / Ryzen 7
- RAM: Minimum 8 GB (16 GB recommended for smooth video processing and deep learning tasks)
- Storage: At least 256 GB SSD for faster access to video datasets and model files
- Graphics Processing Unit (Optional): NVIDIA GPU with CUDA support for accelerating YOLO-V4 training and improving performance
- Display: Standard monitor for development, testing, and visualization of traffic analysis results This configuration allows developers to preprocess video data, train detection models, and evaluate vehicle detection and tracking performance effectively.

Training Hardware Requirements

Training deep learning models typically requires higher computational resources, especially when large traffic video datasets are used. GPU acceleration significantly improves training speed and enables efficient processing of video frames, object detection, and tracking computations.

Typical hardware used for training includes:

- **Processor:** Intel Core i7 / AMD Ryzen 7 or higher.
- **RAM:** 16 GB or higher to handle video data loading, preprocessing, and model training.
- **GPU:** NVIDIA GPU with CUDA support (e.g., RTX 3060 / RTX 4060) for accelerating YOLO-V4 training and Deep SORT computations.
- **Storage:** 512 GB or higher SSD to store video datasets, model weights, and training outputs.

The presence of a GPU helps accelerate tensor operations, convolution processes, and model optimization, ensuring faster and more efficient training of the traffic analysis system.

Deployment Hardware Requirements

After the model is trained, the system can be deployed for real-time traffic analysis using standard computing hardware. Since inference requires fewer resources than training, the system can run efficiently on typical desktop or laptop environments while processing video streams and detecting vehicles.

Typical deployment hardware includes:

- **Processor:** Intel Core i5 or equivalent for handling video processing and detection tasks.
- **RAM:** 8 GB or higher
- **Storage:** 256 GB SSD for application files and model storage
- **GPU:** Optional GPU support for faster prediction
- **Network:** Internet connection for accessing the web interface if deployed online

This configuration enables the system to process traffic videos or live feeds and generate vehicle detection, tracking, and count results with minimal delay.

4.3 Functional Requirements

Functional requirements describe the key operations and capabilities that the proposed road traffic analysis system must perform. These requirements define how the system processes input data, analyzes video frames, detects and tracks vehicles, and generates traffic analysis results.The system follows a structured workflow that begins with video input or live camera feed and ends with the display of detected vehicles, tracking IDs, and total vehicle count.

Video Input and Upload

The system allows users to upload traffic videos or access live camera feeds through an interface. The input data is processed by the backend system for vehicle detection and tracking analysis.

Key functionalities include:

- Accepting traffic videos from users or capturing live video streams through the interface
- Supporting standard video formats such as MP4, AVI, and MOV
- Temporarily storing video frames for processing and real-time analysis
- Validating video input to ensure correct file format and compatibility before processing This functionality provides an easy and reliable way for users to interact with the system.

Video Preprocessing

Before performing traffic analysis, the system preprocesses the input video frames to ensure compatibility with the detection and tracking models. Video preprocessing improves input quality and prepares each frame for accurate model inference.

Preprocessing tasks include:

- Resizing video frames to match the YOLO-V4 model input size
- Normalizing pixel values for efficient deep learning processing
- Converting frames into tensor format for model inference
- Removing noise or distortions from video frames to improve detection accuracy

These preprocessing steps ensure consistent input data for accurate vehicle detection and tracking.

Visualization and Result Display

The system provides visual outputs that help users understand traffic conditions and vehicle movement within the video. Visualization improves interpretability and allows users to verify detection and tracking results effectively.

Main features include:

- Displaying detected vehicles with bounding boxes on video frames
- Showing tracking IDs for each vehicle to monitor movement
- Presenting total vehicle count and traffic analysis results
- Displaying results through a user interface (web or desktop)
- Allowing users to analyze traffic flow and congestion visually

4.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system such as performance, reliability, usability, and security. These requirements ensure that the system operates efficiently and provides a smooth user experience during traffic analysis.

Performance Requirements

The system should process traffic videos efficiently and provide accurate vehicle detection and tracking results in real time. Efficient processing ensures minimal delay in traffic analysis.

- Fast video processing and real-time vehicle detection
- Efficient memory and resource utilization during model inference
- Accurate detection and tracking across different traffic conditions

Scalability

The system should be capable of handling larger traffic datasets and expanding to multiple monitoring locations. Scalability ensures adaptability for future smart city applications.

- Ability to process large-scale traffic video datasets
- Support for multiple camera feeds simultaneously
- Capability to integrate with large-scale traffic monitoring systems

Reliability

Reliability ensures that the system consistently produces accurate and dependable traffic analysis results without failure.

- Stable execution of YOLO-V4 detection and Deep SORT tracking
- Consistent vehicle detection and counting results

Usability

The system should be easy to use and accessible to users with minimal technical knowledge, ensuring effective interaction.

- Simple and intuitive user interface
- Easy video upload or live feed access
- Clear visualization of detected vehicles, tracking IDs, and counts

Security

Security ensures that the system protects user data and input video streams during processing.

- Restricting unsupported video file uploads
- Ensuring safe handling of uploaded videos and data
- Protecting system resources from unauthorized access or misuse

CHAPTER – 5
SYSTEM DESIGN

5.1 Introduction

The system design of the proposed the road traffic analysis using YOLO-V4 & Deep SORT system is an AI-powered solution designed for real-time vehicle detection, tracking, and traffic analysis. By leveraging YOLO-V4 for object detection and Deep SORT for multi-object tracking, the system ensures precise monitoring of vehicle movement, congestion levels, and traffic flow patterns. It processes video footage from CCTV cameras, dashcams, and drone surveillance, providing automated insights to improve road safety, congestion management, and law enforcement monitoring

Purpose of the System Design

The main purpose of the system design is to provide a clear framework for implementing the road traffic analysis system. It ensures that the system processes video footage from CCTV cameras, dashcams, and drone surveillance, providing automated insights to improve road safety, congestion management, and law enforcement monitoring. By integrating AI- driven traffic analysis, it enhances decision-making for urban planning and smart city development, reducing reliance on manual surveillance while improving traffic monitoring efficiency.

- To design a structured flow of data from video input to final traffic analysis output.
- To improve system performance, scalability, and real-time processing capability.
- To provide a clear blueprint for implementation, testing, and future enhancements.
- To ensure efficient utilization of computational resources such as CPU and GPU.

Key Features Integrated into the System Design

The design of the system ensures that the traffic analysis process is efficient, scalable, and adaptable to various traffic environments such as highways, intersections, toll booths, and urban road networks. The system can process both recorded traffic videos and live camera feeds, making it suitable for real-world deployment in intelligent transportation systems.

- Deep learning–based vehicle detection using YOLO-V4 for accurate and real-time object identification.
- Vehicle tracking using Deep SORT to assign unique IDs and track movement across frames.
- Video preprocessing module for frame extraction and preparation of input data.

Technologies Used in System Design

The system is implemented using several technologies that support deep learning, video processing, and application development.

- **Python** for implementing the system logic, algorithms, and overall workflow.
- **YOLO-V4** for real-time vehicle detection in traffic video frames.
- **Deep SORT** for tracking detected vehicles and maintaining unique IDs across frames.
- **TensorFlow / PyTorch** for building and running deep learning models efficiently.
- **OpenCV and NumPy** for video processing, frame extraction, and numerical computations.
- **Matplotlib** for visualization of detection results and traffic analysis outputs.
- **Flask / Tkinter** for developing a user interface to upload videos or display live traffic analysis results.

Security and Data Handling

The system ensures safe handling of input video data and analysis results. Uploaded videos or live camera feeds are used only for traffic analysis and are not permanently stored unless required for evaluation or monitoring purposes. Input validation is applied to accept only supported video formats and maintain data integrity, ensuring secure and reliable processing.

Scalability and Future Expansion

The system design supports future enhancements and improvements. The modular architecture allows additional features to be integrated without major modifications.

- Integration with real-time CCTV camera feeds for continuous traffic monitoring.
- Cloud-based deployment for large-scale traffic analysis and smart city applications.
- Implementation of advanced deep learning models for improved detection accuracy and performance.
- Addition of features such as traffic congestion prediction and automated traffic control systems

5.2 System Architecture

The system architecture describes the structure and interaction between the main components of the Road Traffic Analysis system. The architecture is designed to process traffic videos using deep learning techniques and detect as well as track vehicles present in the scene. The system follows a modular structure consisting of the user interface, backend processing layer, object detection model, tracking module, and output visualization module. These components work together to process input video streams and generate accurate vehicle detection, tracking, and traffic analysis results.

5.2.1 User Interface Layer (Web Interface)

The user interface allows users to interact with the system through a simple web or desktop application. Users can upload traffic videos or access live camera feeds and view the analyzed results generated by the system. The interface is designed to be user-friendly and easy to operate without requiring technical knowledge.

- Video upload functionality through the interface or access to live traffic feed
- Display of detected vehicles with bounding boxes and tracking IDs
- Visualization of vehicle count and traffic analysis results
- Simple and easy interaction for users

Technology Used: HTML, CSS, JavaScript, Flask templates.

5.2.2 Backend Processing Layer – Flask Framework

The backend processing layer handles the core functionality of the system, including video processing, object detection, and tracking operations. It acts as a bridge between the user interface and the deep learning models. The backend receives input video frames, processes them, and sends the results back to the interface.

- Processing of uploaded videos or live camera feeds.
- Frame extraction and preprocessing using OpenCV.
- Integration of YOLO-V4 for object detection.
- Coordination with Deep SORT for vehicle tracking.

5.2.3 YOLO Detection Module

The YOLO-V4 detection module is responsible for identifying vehicles in each frame of the input video. It detects multiple objects in real-time and draws bounding boxes around vehicles such as cars, buses, trucks, and motorcycles.

- Real-time object detection in video frames
- Identification of multiple vehicle types
- Generation of bounding boxes with confidence scores
- High-speed and accurate detection performance

5.2.4 Deep Learning Model

The Deep SORT module is used to track detected vehicles across consecutive frames. It assigns unique IDs to each vehicle and maintains consistent tracking even in crowded traffic conditions.

- Assigning unique IDs to detected vehicles.
- Tracking vehicle movement across frames.
- Preventing duplicate counting of vehicles.
- Maintaining tracking accuracy in dense traffic.

5.2.5 Data Management Layer

The data management layer is responsible for handling input video data, intermediate processing data, and model files required for traffic analysis. It ensures efficient storage, retrieval, and management of data during system operation.

- Handling of input traffic videos and live camera streams.
- Temporary storage of video frames and intermediate processing data.
- Management of YOLO-V4 model weights and Deep SORT tracking files.
- Efficient data flow between processing modules.

5.2.6 Output Visualization Module

The output visualization module displays the final processed results to the user in an understandable format. It shows detected vehicles, tracking IDs, and total vehicle count on the video frames.

- Display of bounding boxes and tracking IDs on vehicles
- Visualization of total vehicle count
- Real-time display of processed video output
- Easy interpretation of traffic conditions.

5.3 System Workflow

The workflow of the proposed road traffic analysis system follows a structured sequence of steps that enables accurate detection, tracking, and counting of vehicles from traffic videos. The system integrates video preprocessing, deep learning model execution, tracking mechanisms, and visualization modules to perform efficient traffic analysis.

Video Input Stage

- Users upload traffic videos or access live camera feeds through the interface
- The system accepts standard video formats such as MP4, AVI, and MOV
- Each video represents a traffic scene containing multiple vehicles
- Input videos or frames are temporarily stored for processing.

Video Preprocessing Stage

- The input video is converted into frames for processing
- Frames are resized to match the YOLO-V4 model input dimensions
- Pixel values are normalized for better model performance
- Frames are converted into tensor format for deep learning processing

Vehicle Detection Stage

- Each processed frame is provided as input to the YOLO-V4 model
- The model detects vehicles such as cars, buses, trucks, and motorcycles
- Bounding boxes are generated around detected vehicles
- Confidence scores are assigned to each detected object

Vehicle Tracking Stage

- Detected vehicles are passed to the Deep SORT tracking module
- Unique IDs are assigned to each vehicle for continuous tracking
- Vehicle movement is tracked across consecutive frames
- Duplicate counting is avoided through consistent tracking

Vehicle Count & Analysis Stage

- The system counts vehicles based on unique tracking IDs
- Traffic flow and density are analyzed from vehicle movement
- Results are accurate for both low and high traffic conditions

Result Visualization Stage

- Detected vehicles with bounding boxes and tracking IDs are displayed
- Total vehicle count and analysis results are shown to the user
- Processed video output is presented through the interface
- Users can visually analyze traffic patterns and congestion

5.4 UML Diagrams

UML (Unified Modeling Language) diagrams are used to visually represent the structure and behavior of a system. They help illustrate how different components interact and how data flows between system modules. In this project, UML diagrams are used to explain the system architecture and workflow of the road traffic analysis system.

The diagram shows the workflow of the proposed road traffic analysis system. An input traffic video or live camera feed is first processed through a preprocessing module where frames are extracted and prepared. The processed frames are then passed to the YOLO-V4 model for vehicle detection. The detected vehicles are further processed by the Deep SORT algorithm for tracking and assigning unique IDs.

The system then performs vehicle counting and traffic analysis based on tracked objects. Finally, the results, including detected vehicles, tracking IDs, and total vehicle count, are displayed through the user interface as the output.

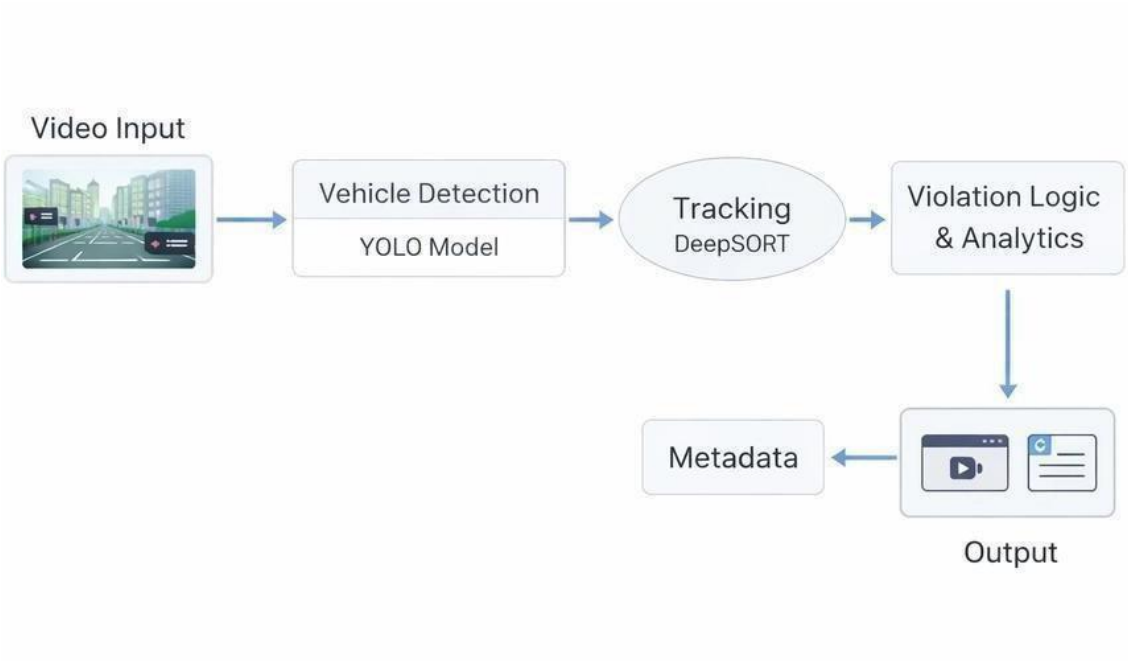

Fig No 5.4.1– Model Workflow

Traffic Analysis Model Architecture

The diagram represents the architecture of the proposed road traffic analysis model. The input traffic video is first divided into frames and processed using the YOLO-V4 network to extract important visual features and detect vehicles. The backbone network (CSPDarknet) extracts spatial features, while neck components such as PANet help in multi-scale feature aggregation for accurate detection of vehicles of different sizes.

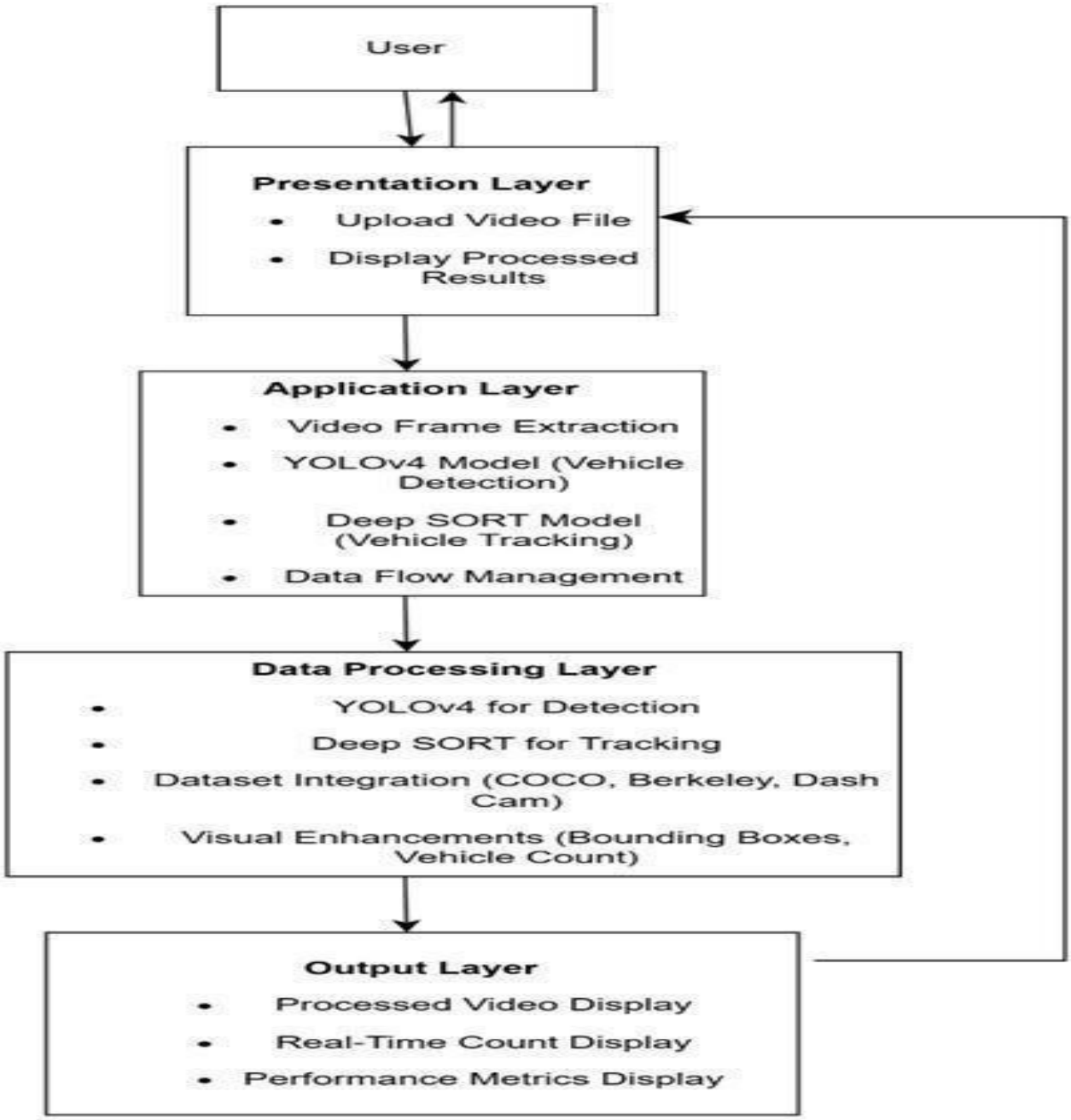

Fig No 5.4.2 – Model Architecture

Image: Use Case Diagram

The use case diagram illustrates the Road Traffic Analysis System with the User interacting through two actions: Upload Video File and View Processed Results. Uploaded videos undergo sequential processing via Video Frame Extraction, YOLOv4 Detection, and Deep SORT Tracking. The results are presented through Display Processed Results, which includes the View Processed Results feature. The user can also access processed results independently. This diagram effectively highlights the system's workflow and user interaction.

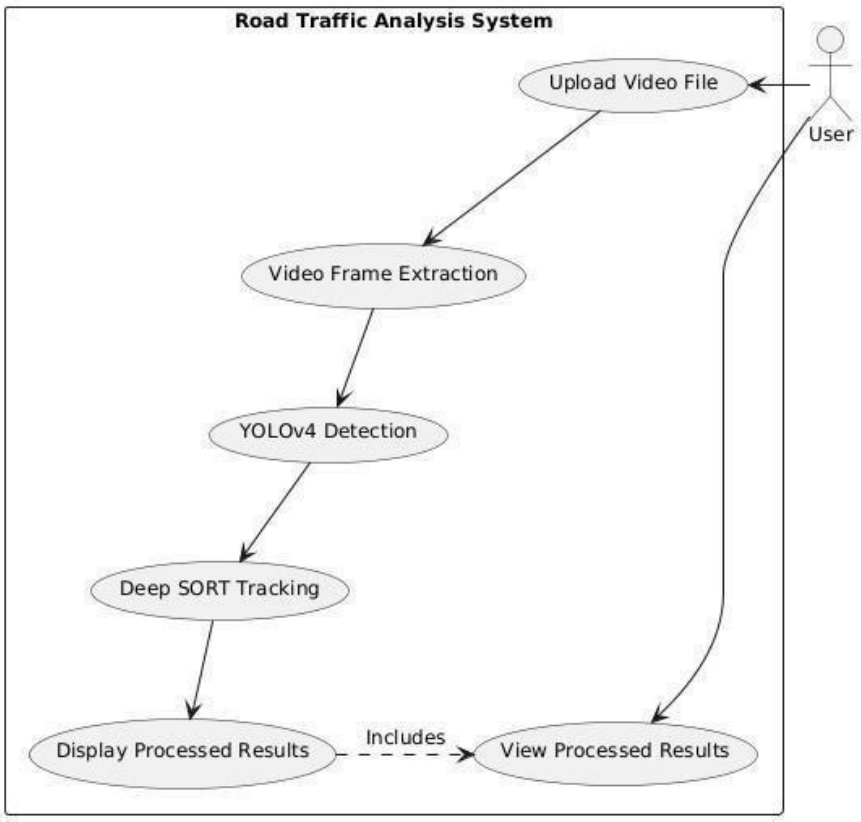

Fig No 5.4.3 – Use case diagram

Image: Class Diagram

The class diagram for the Road Traffic Analysis System outlines key components and their interactions. The Application class launches the system and initializes components. The UserInterface class manages user interactions, including loading models, uploading videos, and displaying results. The Model class handles YOLOv4 and DeepSORT model loading, vehicle detection, and tracking. The VideoProcessor class extracts and preprocesses video frames for efficient analysis. The Result class displays metrics such as vehicle count, FPS, and accuracy for comprehensive insights.

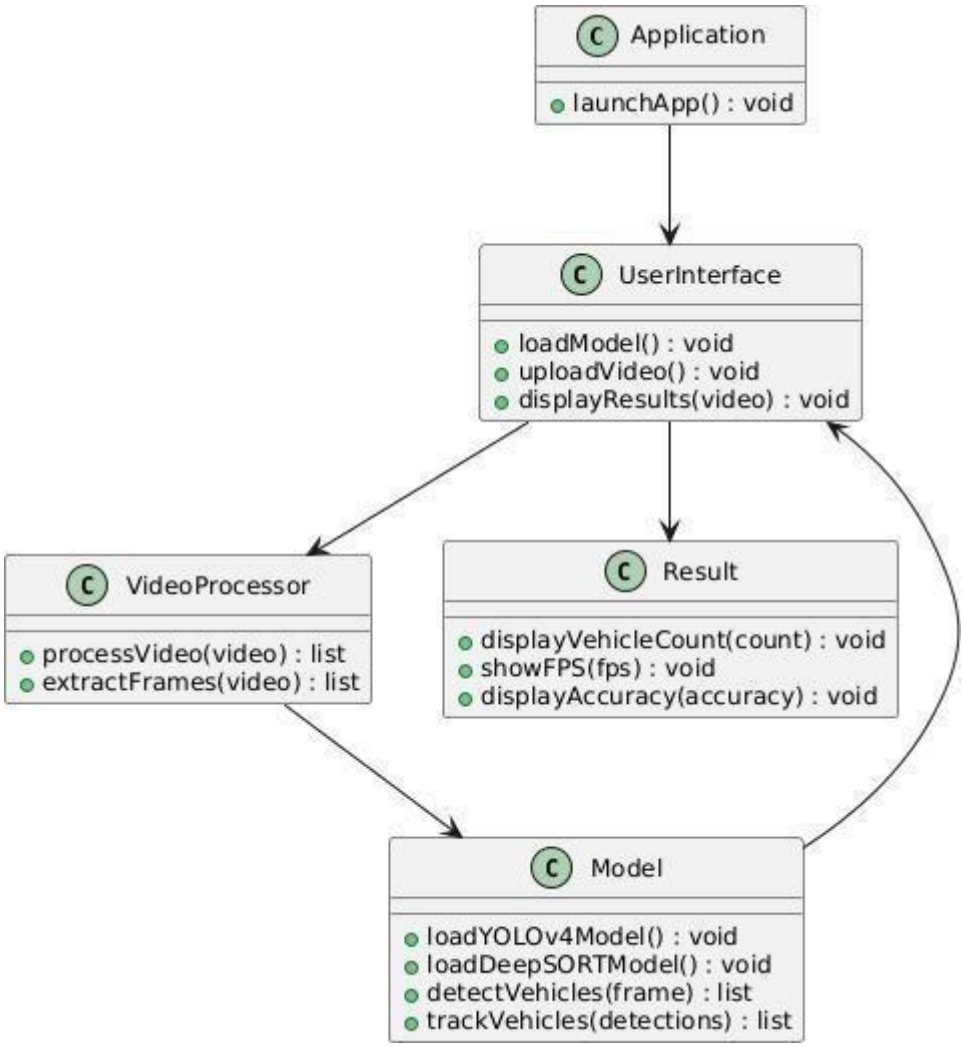

Fig No 5.4.4– Class diagram

Image: Sequence Diagram

The sequence diagram illustrates the workflow of a Road Traffic Analysis system using YOLOv4-Deep SORT and a video processing module. The process starts when the user launches the application, prompting the interface to load the YOLOv4-Deep SORT model. Once the model is successfully loaded, the user uploads a traffic video, which the interface forwards for processing. The video processing module detects and tracks vehicles, generating bounding boxes and tracking IDs. This analyzed data is then returned to the interface, which displays the processed video with tracking information. Finally, the interface presents key metrics such as the vehicle count, FPS, and accuracy, providing the user with comprehensive insights into the traffic analysis.

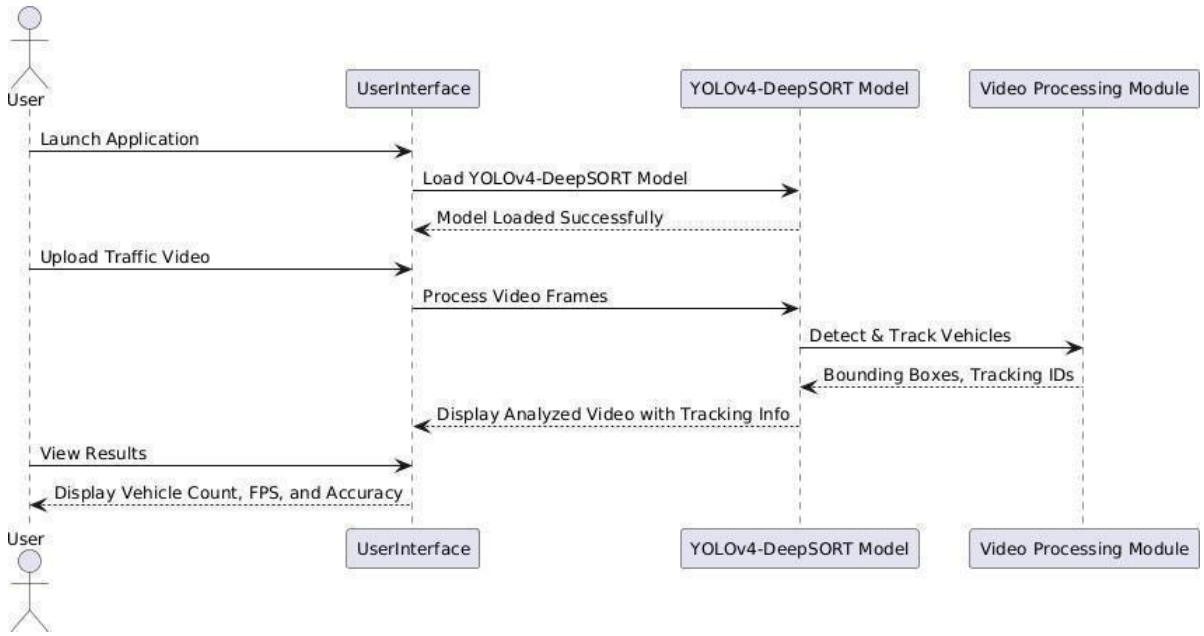

Fig No 5.4.5– Sequence diagram

Image: Activity Diagram

The activity diagram for the Road Traffic Analysis System illustrates the step-by-step workflow of the application. The process begins with launching the application and initializing components. The system loads the YOLOv4-DeepSORT model, then video upload and frame extraction. Each frame undergoes vehicle detection using YOLOv4 and tracking via DeepSORT. The results, including vehicle count, FPS, and accuracy, are then sent to the user interface. The system generates a detailed report to provide comprehensive insights.

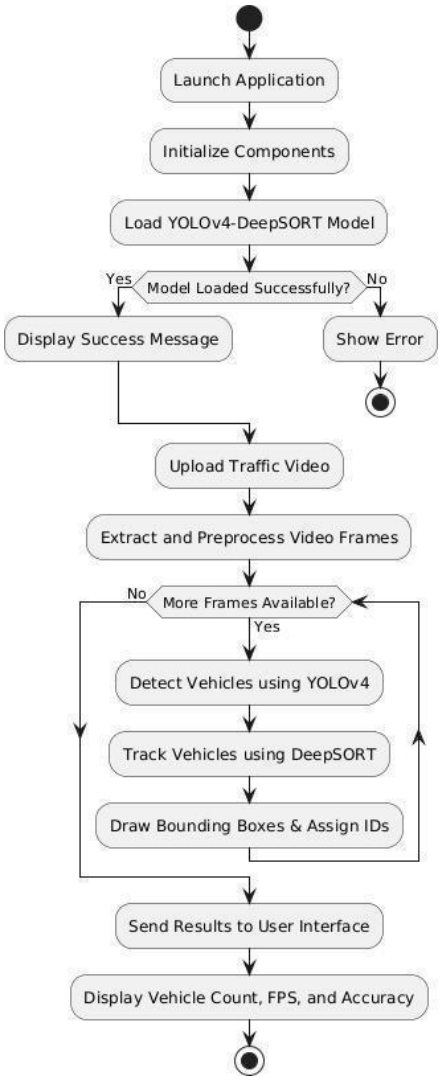

Fig No 5.4.6– Activity diagram

CHAPTER - 6

SYSTEM IMPLEMENTATION

6.1 PROJECT MODULES

The system implementation phase focuses on converting the proposed deep learning architecture for road traffic analysis into a functional application. The implementation includes video preprocessing, vehicle detection, object tracking, vehicle counting, and visualization through a user interface. The system is implemented using Python with deep learning and computer vision libraries to ensure accurate and scalable traffic analysis. The trained models are integrated with a Flask-based application to allow users to upload videos or access live feeds and view traffic analysis results in real time.

6.1.1 Video Preprocessing Module

This module prepares input video data before it is fed into the detection model.

Key Functions:

- Loading traffic videos or capturing live camera feeds
- Extracting frames from video input
- Resizing and normalizing frames for model compatibility
- Converting frames into tensor format for deep learning processing
- Performing preprocessing operations using

OpenCV Technologies Used:

Python, OpenCV, NumPy

6.1.2 Vehicle Detection Module

This module implements the core deep learning model for detecting vehicles.

Key Functions:

- Loading the pre-trained YOLO-V4 model
- Detecting vehicles such as cars, buses, trucks, and motorcycles
- Generating bounding boxes with confidence scores
- Extracting spatial features from video frames Technologies Used: YOLO-V4, TensorFlow / PyTorch

6.1.3 Vehicle Tracking Module

This module tracks detected vehicles across frames.

Key Functions:

- Assigning unique IDs to each detected vehicle
- Tracking vehicle movement across consecutive frames
- Handling occlusions and maintaining tracking consistency
- Preventing duplicate counting of vehicles Technologies Used: Deep SORT, Python

6.1.4 Vehicle Count & Analysis Module

This module calculates vehicle count and analyzes traffic flow.

Key Functions:

- Counting vehicles based on unique tracking IDs
- Analyzing traffic density and flow patterns
- Handling both low and high traffic conditions
- Returning analysis results to the user interface Technologies Used: Python, NumPy

6.1.5 Web Interface & Visualization Module

This module provides a user-friendly interface for system interaction and result display.

Key Functions:

- Allowing users to upload traffic videos or access live feeds
- Displaying detected vehicles with bounding boxes and tracking IDs
- Showing total vehicle count and traffic analysis results
- Enabling real-time visualization of processed video output Technologies Used: Flask, HTML, CSS, javaScript, OpenCV.

The implementation ensures modularity and scalability by separating image preprocessing, deep learning inference, and visualization components. This modular design allows future improvements such as integrating real-time video processing or deploying the model in large-scale surveillance systems.

6.2 ALGORITHMS

The proposed road traffic analysis system utilizes deep learning techniques to detect, track, and analyze vehicles accurately. The primary algorithms used in the system include Convolutional Neural Networks (CNN), YOLO-V4 for object detection, and Deep SORT for tracking.

6.2.1 Convolutional Neural Networks (CNN)

Convolutional Neural Networks are widely used in computer vision tasks due to their ability to automatically learn spatial features from images and video frames.

Working Principle:

- The input video frame passes through multiple convolution layers
- Filters extract important visual features such as edges, shapes, and textures
- Pooling layers reduce spatial dimensions while preserving key information
- The network learns complex patterns representing different vehicle types

Role in the System:

- Extracts meaningful visual features from traffic video frames
- Supports accurate vehicle detection in different traffic conditions
- Handles complex scenes with multiple vehicles

Advantages:

- Automatic feature extraction
- High performance in image and video processing tasks
- Robust to variations in lighting, scale, and vehicle types

6.2.2 YOLO-V4 (You Only Look Once Version 4)

YOLO-V4 is a state-of-the-art object detection algorithm used for real-time detection of vehicles.

Working Principle:

- The input frame is divided into a grid
- Each grid predicts bounding boxes and class probabilities
- The model detects multiple objects in a single forward pass
- Non-Maximum Suppression (NMS) removes duplicate detections

Role in the System:

- Detects vehicles such as cars, buses, trucks, and motorcycles
- Provides bounding boxes with confidence scores
- Enables real-time object detection in traffic videos

Advantages:

- High detection speed and accuracy
- Real-time performance
- Efficient handling of multiple objects in a single frame

6.2.3 Deep SORT (Simple Online and Realtime Tracking)

Deep SORT is a tracking algorithm used to track detected vehicles across frames.

Working Principle:

- Uses motion prediction (Kalman Filter) to estimate object movement
- Applies appearance feature extraction for object matching
- Assigns unique IDs to detected vehicles
- Maintains consistent tracking across frames

Role in the System:

- Tracks vehicles across consecutive frames
- Prevents duplicate counting
- Maintains identity even in occluded conditions

Advantages:

- Accurate multi-object tracking
- Robust to occlusion and motion variations
- Improves counting accuracy

Algorithms and Purpose

- **CNN:** Extracts spatial features from video frames for detection
- **YOLO-V4:** Performs real-time vehicle detection
- **Deep SORT:** Tracks vehicles and maintains unique identities

6.3 PERFORMANCE METRICS

To evaluate the effectiveness of the proposed road traffic analysis system, standard performance metrics are used. These metrics measure how accurately and efficiently the system detects, tracks, and analyzes vehicles.

6.3.1 Accuracy

Accuracy measures how correctly the system detects and classifies vehicles.

Higher accuracy indicates better detection performance

6.3.2 Mean Precision and Recall

- **Precision:** Measures how many detected vehicles are correct
- **Recall:** Measures how many actual vehicles are detected
- Helps evaluate detection reliability

6.3.3 Frames Per Second (FPS)

FPS measures the speed of video processing.

- Higher FPS indicates better real-time performance.

6.3.4 Model Efficiency

Model efficiency evaluates how quickly and effectively the system processes video data.

Key factors include:

- Model inference time
- Memory usage during processing
- Scalability for large traffic datasets

6.4 Screenshots

Screenshots are included to illustrate the working interface and system functionalities of the proposed road traffic analysis system. These screenshots provide a clear understanding of how the system operates in real-time and how users interact with it.

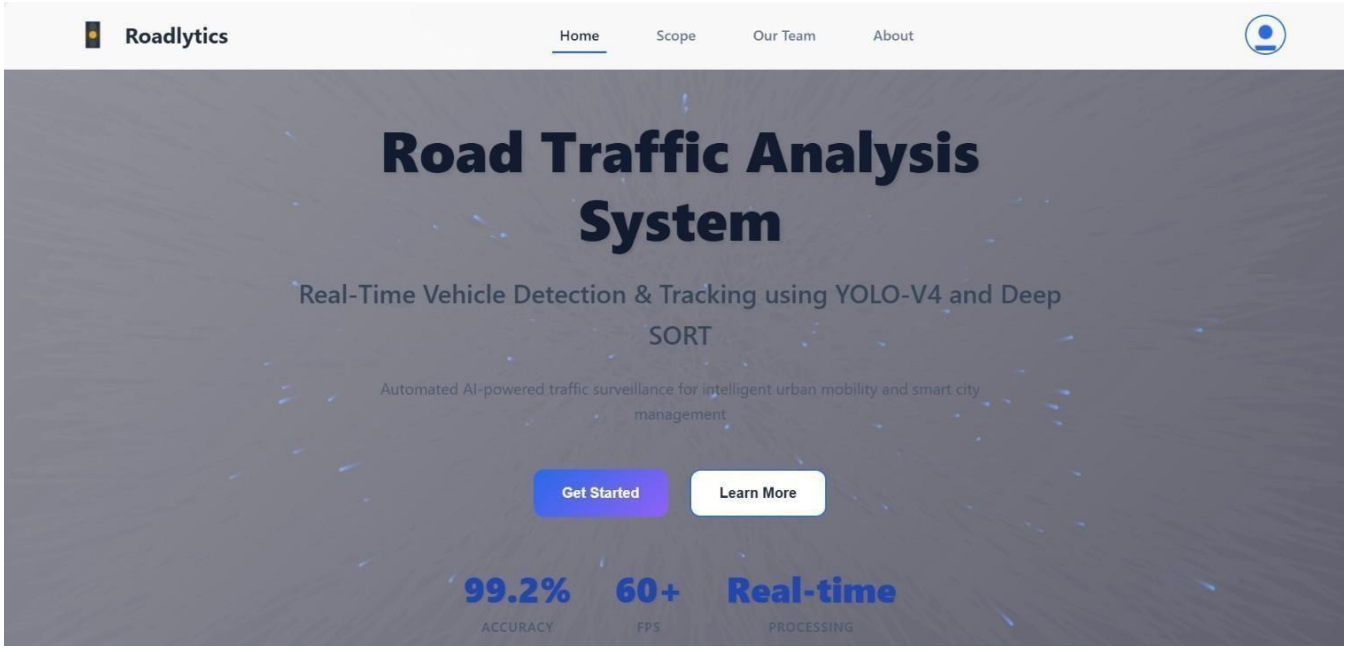

Fig 6.4.1 Home page

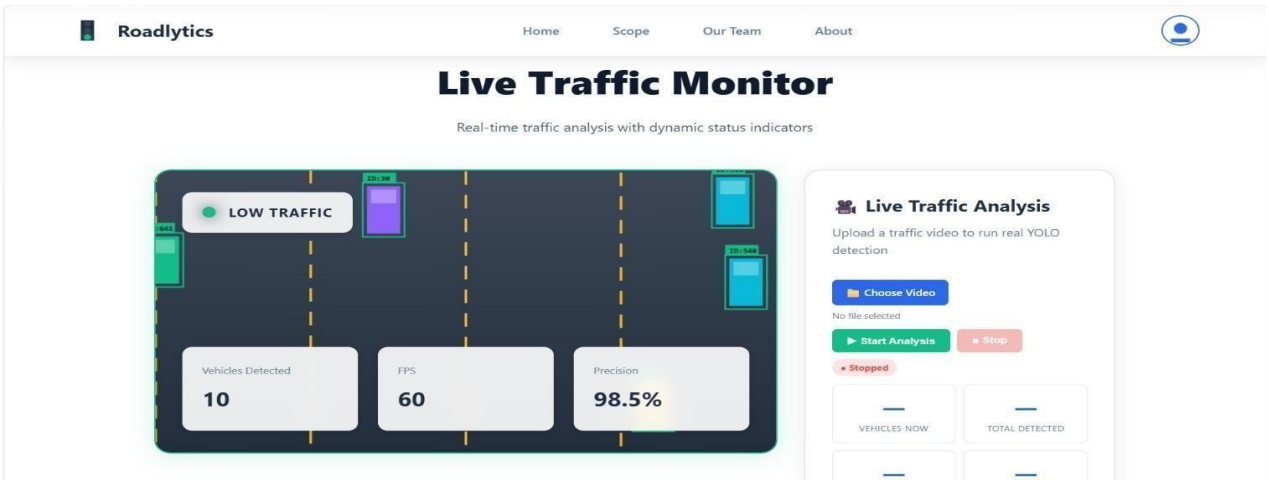

Fig 6.4.2 Analysis page

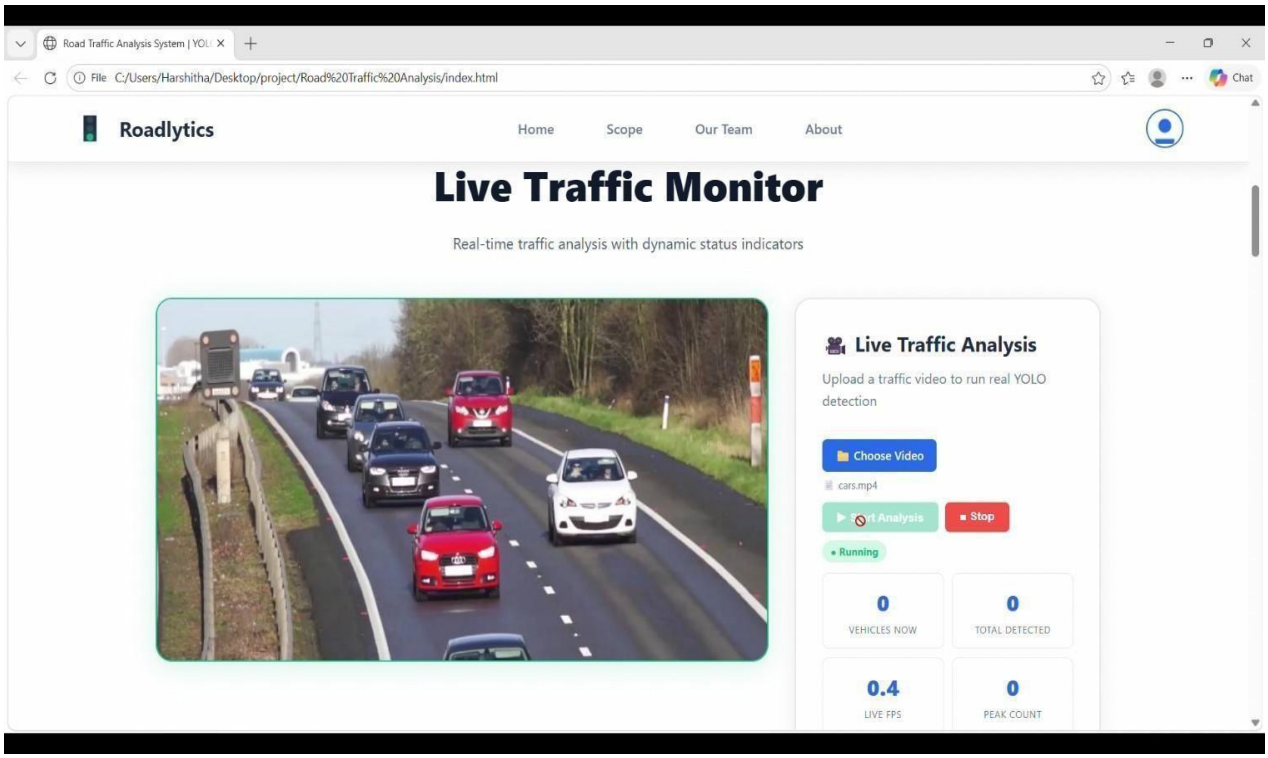

Fig 6.4.3 Traffic Analysis

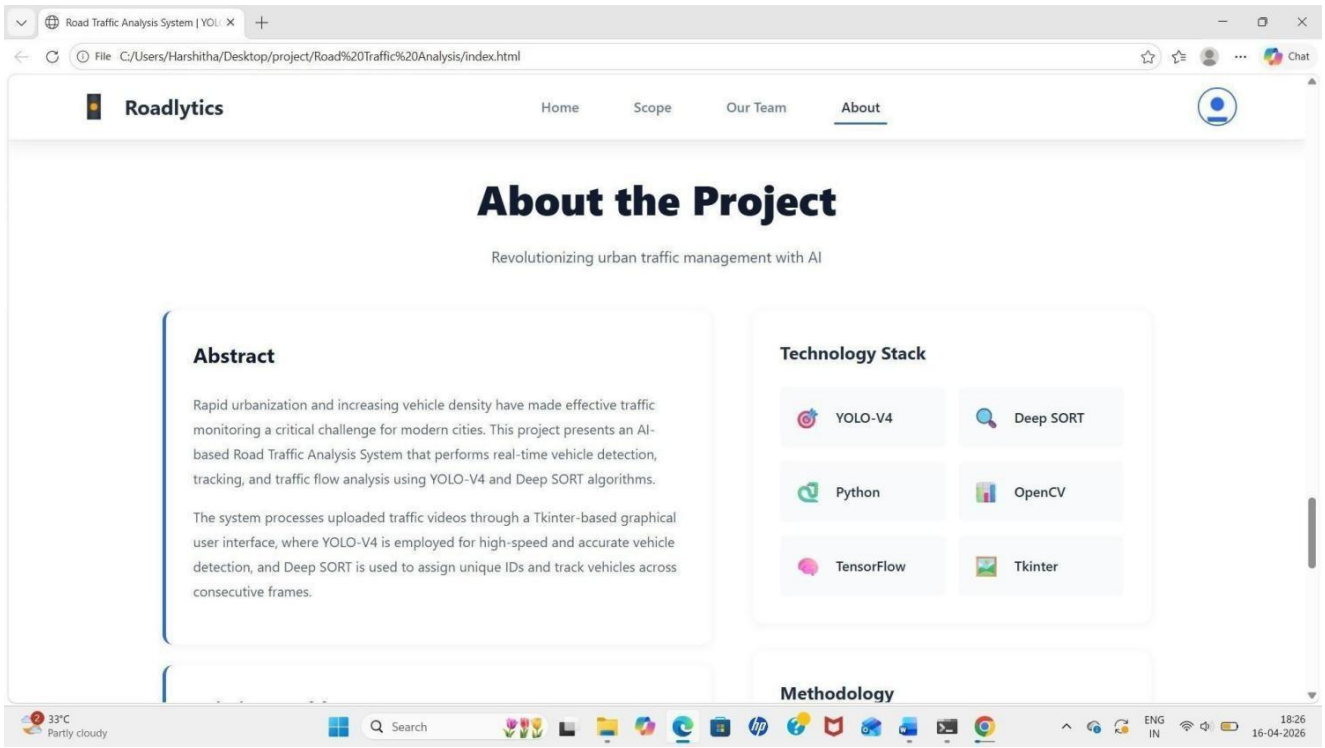

Fig 6.4.4 About Page

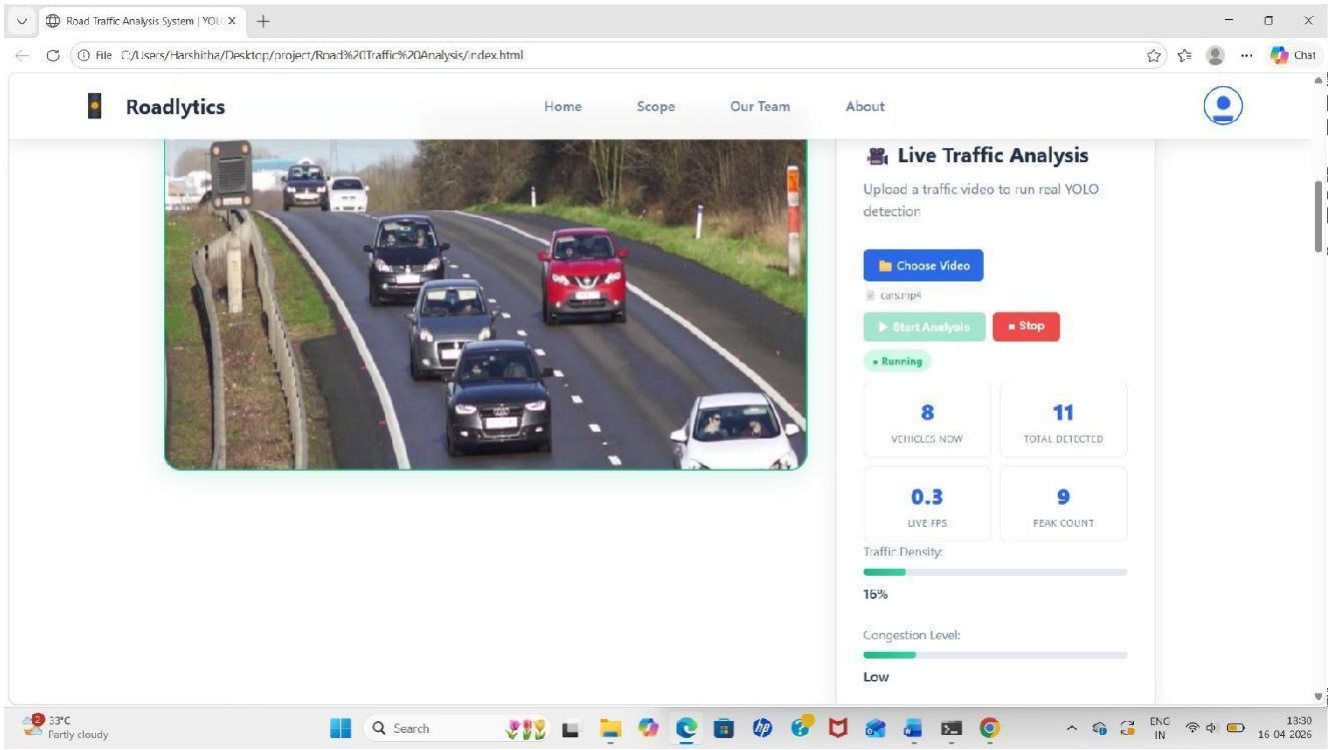

Fig 6.4.5 Live Traffic Analysis

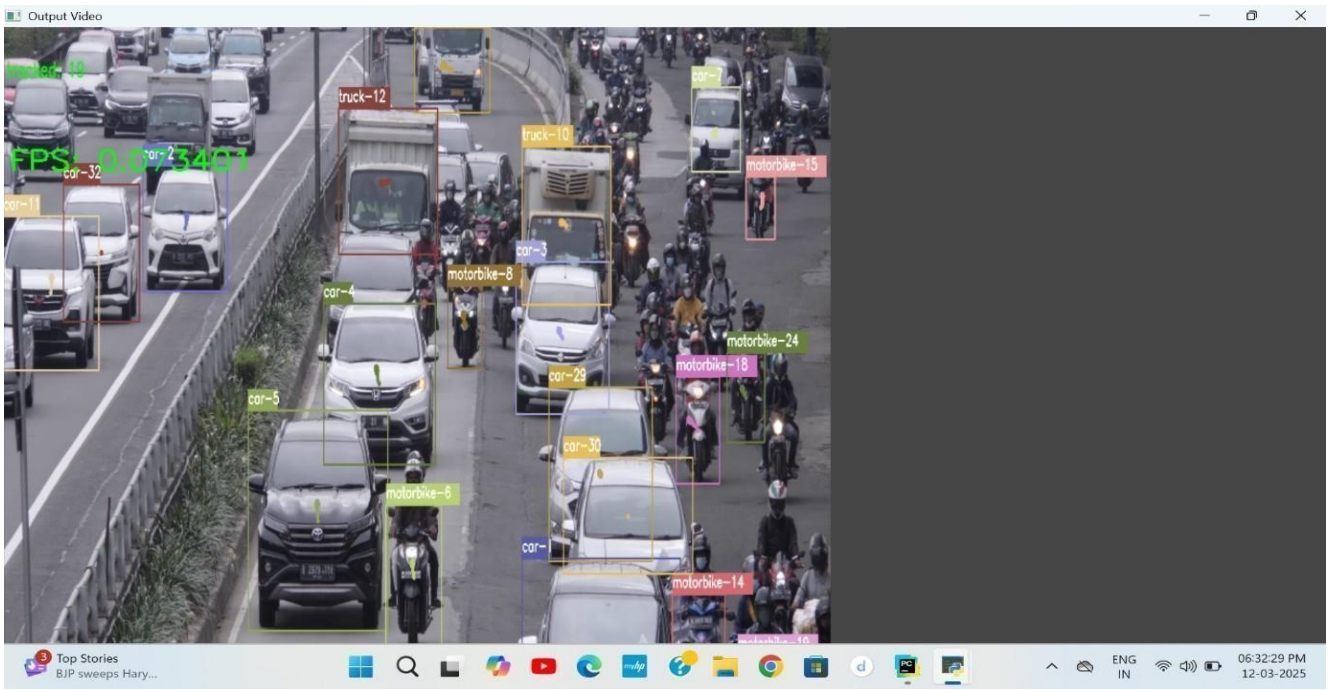

Fig 6.4.6 Result Page

CHAPTER -7
SYSTEM TESTING

7.1 TESTING STRATEGIES

System testing plays an important role in validating the reliability, accuracy, and performance of the proposed Road Traffic Analysis System. The testing phase ensures that the vehicle detection model, video preprocessing pipeline, and tracking modules operate correctly under different traffic conditions. Since traffic analysis systems must produce accurate results for both low and high traffic density scenarios, the system was evaluated using multiple testing strategies focusing on functional correctness, performance efficiency, and integration of deep learning components.

The testing process verifies that the YOLO-V4 model correctly detects vehicles and that the Deep SORT algorithm accurately tracks them across frames. It also confirms that the video preprocessing module, detection pipeline, tracking system, and user interface function correctly and provide accurate traffic analysis results to the user.

To ensure reliability and robustness, the system was evaluated through structured testing stages including unit testing, integration testing, functional testing, performance testing, and user acceptance testing.

7.1.1 Unit Testing

Unit testing focuses on verifying individual components of the road traffic analysis system independently. Each module is tested separately to ensure correct functionality before integrating it into the complete system.

Key Areas Tested

Video Upload Module

- Verified successful upload of traffic videos through the interface.
- Ensured that only supported video formats were accepted.

Video Preprocessing Module

- Tested frame extraction, resizing, and normalization of video frames.
- Verified correct conversion of frames into tensor format for model processing.

Model Loading Module

- Verified successful loading of the YOLO-V4 model and Deep SORT tracking model.
- Ensured the models were properly initialized and ready for detection and tracking.

Detection & Tracking Module

- Tested accurate detection of vehicles in video frames using YOLO-V4.
- Verified correct assignment of unique IDs and tracking using Deep SORT.

7.1.2 Integration Testing

Integration testing ensures that all modules of the road traffic analysis system work correctly when combined together.

Test Scenarios

Video Processing Pipeline Integration

- Verified that uploaded traffic videos or live feeds were correctly passed to the preprocessing module.

Model Integration

- Ensured that the preprocessed video frames were successfully processed by the YOLO-V4 detection model and Deep SORT tracking module.

Backend–Frontend Communication

- Tested the interaction between the Flask backend server and the web interface.

Prediction Workflow

- Confirmed that vehicle detection, tracking, and counting results were correctly processed and displayed to the user.

Integration testing ensured smooth communication between system components and reliable traffic analysis results.

7.1.3 Functional Testing

Functional testing verifies that the road traffic analysis system behaves according to the specified functional requirements.

Tested Features

Video Upload Functionality

- Verified successful uploading of traffic videos through the interface

Vehicle Detection

- Confirmed that the system correctly detected vehicles such as cars, buses, trucks, and motorcycles
- Verified that vehicles are highlighted using green bounding boxes

Vehicle Tracking

- Ensured that each detected vehicle is assigned a unique ID using Deep SORT
- Verified continuous tracking across multiple frames

Vehicle Counting

- Tested accurate counting of vehicles without duplication

Result Display

- Verified correct visualization of detection results, tracking IDs, and total vehicle count on the interface

7.1.4 Performance Testing

Performance testing evaluates the computational efficiency of the road traffic analysis system.

Key Metrics Evaluated

Processing Time

- Measured the time taken by the system to process video frames and detect vehicles

Model Inference Speed

- Evaluated the performance of the YOLO-V4 model during real-time vehicle detection

Tracking Efficiency

- Assessed the performance of the Deep SORT algorithm in tracking multiple vehicles across frames

System Responsiveness

- Measured the response time of the interface when processing video inputs and displaying results

Scalability

- Tested the system's ability to handle multiple video streams or large datasets without performance degradation

7.1.5 User Acceptance Testing (UAT)

User Acceptance Testing was conducted to evaluate the usability and effectiveness of the road traffic analysis system from the user's perspective.

Key Evaluation Criteria

Ease of Use

- Verified that the user interface was simple, intuitive, and easy to operate

Prediction Accuracy

- Confirmed that the system provided reliable vehicle detection, tracking, and counting results

Visualization Quality

- Ensured that vehicles highlighted with green bounding boxes, tracking IDs, and counts were clearly visible

Workflow Efficiency

- Evaluated the ease of uploading videos or accessing live feeds and viewing results

User Feedback

- The deep learning–based approach provided accurate vehicle detection and tracking results
- The visualization using bounding boxes and tracking IDs improved understanding of traffic flow
- The system effectively handled both low and high traffic density scenarios
- Future improvements may include real-time multi-camera integration and advanced traffic prediction

7.2 TESTING METHODOLOGIES

The road traffic analysis system uses multiple testing methodologies to ensure the correctness, reliability, and performance of the implemented detection, tracking, and web application modules.

7.2.1 Black Box Testing

Black box testing evaluates system outputs without considering internal implementation details.

Key Areas Tested

Video Upload

- Valid traffic video formats were accepted while invalid files were rejected

Crowd Prediction

- Verified that the system produced accurate vehicle detection and counting results

Web Interface Output

- Ensured that detection results, tracking IDs, and vehicle counts were correctly displayed

Outcome

The system produced correct outputs and handled invalid inputs effectively.

7.2.2 White Box Testing

White box testing examines the internal implementation and logic of the system.

Key Areas Tested

Video Preprocessing Pipeline

- Verified correct frame extraction, resizing, and normalization operations

Model Execution

- Tested internal working of YOLO-V4 detection and Deep SORT tracking modules

Tracking & Counting Logic

- Confirmed correct assignment of IDs and accurate vehicle counting

Outcome

Internal modules performed efficiently and produced consistent results.

7.2.3 Regression Testing

Regression testing ensures that updates or improvements to the system do not affect existing functionalities.

Key Areas Tested

- Model updates and parameter tuning
- Changes in preprocessing operations
- Improvements to the user interface

Outcome

System updates did not affect detection accuracy or system stability.

7.2.4 Boundary Value Analysis

Boundary value testing evaluates system performance under extreme input conditions.

Test Cases

- Low traffic scenarios with very few vehicles
- Extremely dense traffic conditions
- Videos with varying resolutions and frame rate

Outcome

The system successfully handled different traffic densities without failure.

7.2.5 Error Handling Testing

This testing ensures that unexpected inputs are handled correctly.

Test Cases

- Invalid video formats.
- Corrupted video files.

Outcome

The system generated appropriate error responses and prevented crashes.

7.2.6 Load Testing

Load testing evaluates system performance under heavy workloads.

Test Scenarios

- Processing multiple traffic videos simultaneously.
- Continuous real-time analysis through the interface

Outcome

The system maintained stable performance under increased workload conditions.

7.3 Test Cases

Test Case ID	Module	Input	Expected Output	Status
TC01	Video Upload	Valid traffic video	Video uploaded successfully	Pass
TC02	Video Upload	Invalid file format	Upload rejected with message	Pass
TC03	Preprocessing	High resolution video	Frames resized and normalized	Pass
TC04	Preprocessing	Low resolution video	Frames processed correctly	Pass
TC05	Model Loading	YOLO-V4 model	Model loaded successfully	Pass
TC06	Detection	Traffic video frame	Vehicles detected accurately	Pass
TC07	Tracking	Detected vehicles	Unique IDs assigned	Pass
TC08	Tracking	Multiple frames	Vehicles tracked continuously	Pass
TC09	Counting	Sparse traffic	Accurate vehicle count	Pass
TC10	Counting	Dense traffic	Vehicle count predicted correctly	Pass

TC11	Visualization	Detection output	Bounding boxes displayed correctly	Pass
TC12	Visualization	Tracking output	Tracking IDs displayed	Pass
TC13	Interface	Prediction result	Vehicle countshown on interface	Pass
TC14	System	Multiple videos	Videos processed withouterror	Pass
TC15	System	Large video dataset	Processed successfully	Pass
TC16	Integration	Full workflow	End-to-end trafficanalysis works	Pass
TC17	Performance	High traffic video	Processing within acceptabletime	Pass
TC18	Error Handling	Corrupted video file	Error handled safely	Pass
TC19	Data Handling	Different Traffic densities	Consistent detection and tracking	Pass
TC20	Scalability	Multiple requests	System remains stable	Pass
TC21	Logging	Detection event	Results logged successfully	Pass

TC22	Stress Test	Extremely dense traffic	System remains stable	Pass
TC23	Model Accuracy	Testvideo dataset	Accurate detection results	Pass
TC24	Model Evaluation	Validation dataset	Accuracy metrics calculated	Pass
TC25	Deployment	Real-time video feed	Instant traffic analysis output	Pass

CHAPTER - 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

The developed road traffic analysis system using deep learning successfully demonstrates the effectiveness of real-time vehicle detection and tracking techniques for traffic monitoring and analysis. The implementation follows a structured workflow including video preprocessing, object detection, vehicle tracking, and traffic analysis using YOLO-V4 and Deep SORT algorithms. Unlike traditional methods that rely on manual observation or basic image processing, the proposed system automatically detects and tracks multiple vehicles with high accuracy and speed.

The integration of the YOLO-V4 model enables efficient and accurate detection of various vehicle types, while the Deep SORT algorithm ensures reliable tracking by assigning unique IDs to each vehicle. This combination allows the system to handle challenges such as occlusion, varying traffic density, and complex road environments. The system also incorporates a user interface that allows users to upload traffic videos or access live feeds and view analysis results in real time.

Experimental results demonstrate that the proposed system provides accurate vehicle detection and counting across both low and high traffic scenarios. The modular architecture ensures scalability, maintainability, and ease of deployment for practical applications such as traffic monitoring, congestion control, smart city systems, and road safety management.

Overall, the system successfully achieves the objective of developing an efficient and scalable traffic analysis solution using deep learning techniques.

- The proposed project, *"Road Traffic Analysis using YOLO-V4 & Deep SORT,"* effectively addresses challenges in monitoring and analyzing traffic in complex environments.
- The use of deep learning techniques improves the accuracy and efficiency of vehicle detection and tracking compared to traditional methods.
- The YOLO-V4 model combined with Deep SORT enables real-time detection and reliable multi-object tracking.
- The system accurately counts vehicles and analyzes traffic flow even in dense and congested conditions.
- The user interface provides an interactive platform for uploading videos and visualizing analysis results.
- The system demonstrates strong performance and reliability for real-world traffic monitoring applications.

8.2 SCOPE OF FUTURE ENHANCEMENTS

Although the proposed road traffic analysis system performs effectively, several improvements can further enhance its accuracy, scalability, and real-time capabilities. Future research can explore advanced deep learning architectures such as transformer-based vision models or attention-based networks for improved vehicle detection and traffic pattern analysis. Integration of real-time multi-camera systems can enable continuous traffic monitoring across multiple locations for intelligent transportation systems.

Deployment of the system using cloud platforms such as AWS, Google Cloud, or Azure can improve scalability and enable large-scale traffic monitoring. Real-time video processing using advanced streaming technologies can further extend the system for smart city applications. Additionally, incorporating explainable AI techniques can improve transparency and understanding of model predictions. These enhancements will help transform the system into a comprehensive intelligent traffic management platform suitable for real-world deployment.

Although the proposed system performs effectively, several improvements can further enhance its capabilities:

- Real-Time Traffic Monitoring: Integration with live CCTV feeds for continuous traffic analysis
- Advanced Deep Learning Models: Implementation of transformer-based or attention-based models for improved detection accuracy
- Cloud Deployment: Deployment on cloud infrastructure for large-scale traffic monitoring and data management
- Mobile Integration: Development of mobile applications for accessing traffic analysis and monitoring remotely
- Improved Visualization: Enhanced visualization dashboards for better understanding of traffic flow and congestion
- Smart City Integration: Integration with smart traffic control systems for automated signal management and road safety

APPENDIX: SOURCE CODE

The proposed road traffic analysis system is implemented using Python and deep learning frameworks. Input traffic videos or live camera feeds are first preprocessed and converted into frames, which are then passed to the YOLO-V4 model for vehicle detection. The detected vehicles are further processed by the Deep SORT algorithm for tracking and assigning unique IDs. The total vehicle count is obtained based on tracked objects, and the results are visualized through the user interface for effective traffic analysis.

```
import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename
from tkinter import ttk
import numpy as np
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"  # Disable GPU
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0" # Disable oneDNN optimizations
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'  # Suppress TensorFlow warnings
import time
import tensorflow as tf
physical_devices = tf.config.experimental.list_physical_devices('GPU')
if len(physical_devices) > 0:
tf.config.experimental.set_memory_growth(physical_devices[0], True)
from absl import app, flags, logging
from absl.flags import FLAGS
import core.utils as utils
from core.yolov4 import filter_boxes
from tensorflow.python.saved_model import tag_constants
from core.config import cfg
from PIL import Image
import cv2
```

```
import matplotlib.pyplot as plt
from tensorflow.compat.v1 import ConfigProto
from tensorflow.compat.v1 import InteractiveSession
# deep sort imports
from deep_sort import preprocessing, nn_matching
from deep_sort.detection import Detection
from deep_sort.tracker import Tracker
import sys
sys.path.insert(0, os.getcwd())
import tools.generate_detections as gdet
from tqdm import tqdm
from collections import deque
pts = [deque(maxlen=30) for _ in range(9999)]
main = tkinter.Tk()
main.title("Road Traffic Analysis using YOLO-V4 & Deep Sort")
main.geometry("1300x1200")
from PIL import Image, ImageTk
bg_image = Image.open("traffic.jpg")
bg_image = bg_image.resize((900, 400))
bg_photo = ImageTk.PhotoImage(bg_image)
bg_label = Label(main, image=bg_photo)
bg_label.place(x=0, y=0, relwidth=1, relheight=1)
global filename
global model, encoder, tracker, config
max_cosine_distance = 0.4
nn_budget = None
nms_max_overlap = 1.0
global accuracy, precision
def loadModel():
    global encoder, tracker, config
    model_filename = os.path.join(os.getcwd(), "model_data", "mars-small128.pb")
    if not os.path.exists(model_filename):
```

```
messagebox.showerror("Error", "DeepSORT model not found:\n" + model_filename)
  return
    encoder = gdet.create_box_encoder(model_filename, batch_size=1)
    metric = nn_matching.NearestNeighborDistanceMetric(
        "cosine", max_cosine_distance, nn_budget)
    tracker = Tracker(metric)
    config = ConfigProto()
    config.gpu_options.allow_growth = True
    session = InteractiveSession(config=config)
    pathlabel.config(text="YOLOv4 DeepSort Model Loaded")
    text.delete('1.0', END)
    text.insert(END, "YOLOv4 + DeepSORT Model Loaded Successfully\n\n")
def vehicleDetection():
    global encoder
    if 'encoder' not in globals():
        messagebox.showerror("Error", "Please load the model first by clicking 'Generate & Load
YOLOv4-DeepSort Model'")
        return
    global model, encoder, tracker
    filename = filedialog.askopenfilename(
        initialdir=os.path.join(os.getcwd(), "data", "video"),
        title="Select Traffic Video",
        filetypes=(("MP4 files", "*.mp4"), ("All files", "*.*")))
    if filename == "":
        messagebox.showwarning("Warning", "No video selected")
        return
    pathlabel.config(text=filename)
    text.delete('1.0', END)
    text.insert(END, filename + " loaded\n\n")
    # Load YOLO model
    saved_model_loaded =
        tf.saved_model.load( 'yolo/yolov4-416',
        tags=[tag_constants.SERVING])
```

```
model = saved_model_loaded.signatures['serving_default']
cap = cv2.VideoCapture(filename)
if not cap.isOpened():
    messagebox.showerror("Error", "Could not open video")
    return
# OPT 1: Increase frame skip to reduce processing load
frame_skip = 5
frame_count = 0
# Input size must match the saved model (yolov4-416 expects 416x416)
INPUT_SIZE = 416
# OPT 3: Track FPS for monitoring
fps_start = time.time()
fps_counter = 0
while True:
    ret, frame = cap.read()
    if not ret:
        print("Video ended or frame not received")
        break
    frame_count += 1
    if frame_count % frame_skip != 0:
        continue # OPT 4: Resize early to reduce memory load throughout the pipeline
    frame = cv2.resize(frame, (640, 480))
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    image_data = cv2.resize(frame, (INPUT_SIZE, INPUT_SIZE))
    image_data = image_data / 255.
    image_data = image_data[np.newaxis, ...].astype(np.float32)
    batch_data = tf.constant(image_data)
    pred_bbox = model(batch_data)
    for key, value in pred_bbox.items():
        boxes = value[:, :, 0:4]
        pred_conf = value[:, :, 4:] # OPT 5: Reduce max detections from 50 to 30 for faster NMS
    boxes, scores, classes, valid_detections = tf.image.combined_non_max_suppression(
```

```
 boxes=tf.reshape(boxes, (tf.shape(boxes)[0], -1, 1, 4)),
scores=tf.reshape(pred_conf, (tf.shape(pred_conf)[0], -1, tf.shape(pred_conf)[-1])),
max_output_size_per_class=30,
  max_total_size=30,
 iou_threshold=0.45,
 score_threshold=0.50)
      num_objects = valid_detections.numpy()[0]
        bboxes = boxes.numpy()[0][:int(num_objects)]
        scores_np = scores.numpy()[0][:int(num_objects)]
        classes_np = classes.numpy()[0][:int(num_objects)]
        original_h, original_w, _ = frame.shape
        bboxes = utils.format_boxes(bboxes, original_h, original_w)
        features = encoder(frame, bboxes)
        detections = [
            Detection(bbox, score, "vehicle", feature)
            for bbox, score, feature in zip(bboxes, scores_np, features) ]
        tracker.predict()
        tracker.update(detections)
        for track in tracker.tracks:
            if not track.is_confirmed() or track.time_since_update > 1:
                continue
            bbox = track.to_tlbr()
            cv2.rectangle(
                frame,
                (int(bbox[0]), int(bbox[1])),
                (int(bbox[2]), int(bbox[3])),
                (0, 255, 0),
                2)# OPT 6: Show track ID on each vehicle
            cv2.putText(
frame,
f"ID: {track.track_id}",
(int(bbox[0]), int(bbox[1]) - 10),
```

```
 cv2.FONT_HERSHEY_SIMPLEX,
0.5,
 (0, 255, 0),
2 ) # OPT 7: Display live FPS on frame
fps_counter += 1
  elapsed = time.time() - fps_start
        if elapsed > 0:
            fps = fps_counter / elapsed
            cv2.putText(
                frame,
                f"FPS: {fps:.1f}",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                1,
                (0, 0, 255),
                2 )
        cv2.imshow("Traffic Detection", cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))
        # OPT 8: waitKey(1) keeps display responsive without blocking
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
def close():
main.destroy()
font = ('times', 20, 'bold')
title = Label(main, text='Road Traffic Analysis using YOLO-V4 & Deep Sort', bg='#fdae61',
fg='black', font=('Times New Roman', 30, 'bold'))
title.place(relx=0.5, y=10, anchor='n')
font1 = ('times', 14, 'bold')
font2 = ('times', 12, 'bold')
def display_content(content):
```

```
text_box.delete('1.0', END)
text_box.insert(END, content)
def show_default_message():
display_content(why_choose_us_content)
```

CHAPTER -9

CONFERENCE PROCEEDINGS

The research work titled *"Real-Time Road Traffic Analysis using YOLO-V4 and Deep SORT"* was prepared for academic and research presentation.

- The project focuses on applying deep learning–based vehicle detection and tracking techniques for accurate traffic analysis and monitoring.
- The methodology, system architecture, and experimental evaluation were documented in a structured conference paper format following standard IEEE/ACM research guidelines.
- The proposed YOLO-V4 model combined with Deep SORT tracking was highlighted as the core contribution for accurate vehicle detection and multi-object tracking.
- A comparative study with traditional traffic monitoring approaches such as manual counting and basic computer vision techniques was conducted to validate improvements in accuracy and efficiency.
- The research explains the process of video preprocessing, object detection, feature extraction, and vehicle tracking using deep learning techniques.
- The system architecture and workflow demonstrate how traffic video frames are processed through the trained model to detect, track, and count vehicles.
- Performance evaluation metrics such as accuracy, precision, recall, and processing speed (FPS) were used to assess system performance.
- The effectiveness of the system in handling dense traffic conditions and occlusions was discussed in the experimental analysis.
- The research highlights how deep learning models automatically learn spatial and temporal features from traffic videos for improved traffic analysis.
- Scalability analysis was conducted to evaluate the system's capability to process large-scale traffic data efficiently.
- The system's applicability for real-world scenarios such as traffic monitoring, congestion control, and smart city infrastructure was emphasized.
- Practical considerations related to video data processing, model deployment, and system performance optimization were also discussed.

CHAPTER-10

BIBLIOGRAPHY

[1] Bochkovskiy, A., Wang, C., & Liao, H. Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934. Describes YOLOv4's architecture, performance optimizations, and its application in real-time vehicle detection.

[2] Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric*. IEEE International Conference on Image Processing (ICIP). Explains the Deep SORT algorithm used for robust multi-object tracking in the project.

[3] Tkinter Documentation. (2023). *Python's Tkinter Standard Library*. Retrieved from https://docs.python.org/3/library/tkinter.html. Details Tkinter's role in GUI development, including layout design and event handling used in the project's interface.

[4] TensorFlow Documentation. (2023). *TensorFlow API Guide*. Retrieved from https://www.tensorflow.org/. Provides details on TensorFlow's saved model API, enabling the YOLOv4 model's deployment and performance tuning.

[5] OpenCV Documentation. (2023). *OpenCV Library for Computer Vision*. Retrieved from https://opencv.org/. Describes key image processing techniques applied in video frame extraction, object detection refinement, and improved visual output.

[6] PIL (Pillow) Documentation. (2023). *PIL (Python Imaging Library) for Image Processing*. Retrieved from https://pillow.readthedocs.io/. Guides the PIL library used for image resizing, conversion, and visualization in the Tkinter UI.

[7] JetBrains Documentation. (2023). *PyCharm IDE Guide*. Retrieved from https://www.jetbrains.com/pycharm/. Explains PyCharm's debugging tools, project structure management, and Python development best practices.

[8] tqdm Documentation. (2023). *tqdm: A Fast, Extensible Progress Bar for Python*. Retrieved from https://tqdm.github.io/. Provides insights on integrating progress bars for improved user feedback during traffic analysis.

PAPER PUBLICATION

International Journal of All Research Education and Scientific Methods (IJARESM),
ISSN: 2455-6211, Volume 14, Issue 4, April-2026, Available online at: www.ijaresm.com

Road Traffic Analysis Using YOLOv4 and Deep SORT

Mrs. P. Seshu Kumari[1], Sandaka Venkata Rajesh[2], Atapakala Devi Harshitha[3], Leela Prasanna Yeleti[4], Sangadi Maharaju[5]

[1] Assistant Professor, Department of Computer Science and Engineering (AI), Pragati Engineering College, ADB Road, Surampalem, Near Kakinada, East Godavari District, Andhra Pradesh, India-533437
[2,3,4,5] B.Tech Students, Department of Computer Science and Engineering(AI), Pragati Engineering College, ADB Road, Surampalem, Near Kakinada, East Godavari District, Andhra Pradesh, India-533437

DOI: https://doi.org/10.56025/IJARESM.140426361

ABSTRACT

The rapid growth of urban populations and vehicular density has led to significant challenges in traffic management, including congestion, increased travel time, and environmental pollution. Traditional traffic monitoring systems rely on manual surveillance and sensor-based approaches, which are inefficient, costly, and lack scalability. To address these limitations, this research proposes an AI-powered road traffic analysis system using YOLOv4 for real-time vehicle detection and Deep SORT for multi-object tracking. The system processes live and recorded video streams from traffic cameras to detect and track vehicles such as cars, buses, trucks, and motorcycles. Each detected vehicle is assigned a unique ID, enabling accurate tracking across frames and preventing duplicate counting. Advanced preprocessing techniques, including frame extraction, normalization, and bounding box filtering, improve detection accuracy and performance. The system is implemented using Python, TensorFlow/PyTorch, OpenCV, and NumPy, ensuring efficient real-time processing. It provides insights into traffic density, vehicle movement patterns, and congestion hotspots. The proposed system enhances decision-making for traffic authorities, enabling optimized traffic flow, improved road safety, and support for smart city initiatives. Future work includes integration with IoT, predictive traffic analytics, and cloud-based deployment.

Keywords: Traffic Analysis, YOLOv4, Deep SORT, Vehicle Detection, Object Tracking, Computer Vision, Smart Cities, Deep Learning.

1. INTRODUCTION

Traffic congestion has become a critical issue in modern urban environments, resulting in delays, increased fuel consumption, and environmental pollution. Traditional traffic monitoring systems rely on manual observation, static sensors, and fixed signal timings, which are not capable of adapting to dynamic traffic conditions. Recent advancements in artificial intelligence and deep learning have introduced intelligent approaches for real-time traffic monitoring. The integration of YOLOv4 for object detection and Deep SORT for tracking provides a scalable and automated solution for traffic analysis.The proposed system processes video streams from traffic cameras to detect and track vehicles, analyze traffic patterns, and identify congestion hotspots. This enables authorities to make data-driven decisions for efficient traffic management and improved urban mobility.

1.1 Problem Statement

Existing traffic monitoring systems suffer from:

- Manual monitoring and human dependency
- High installation and maintenance cost
- Lack of real-time analysis
- Inability to handle high traffic density
- Limited accuracy in tracking and counting vehicles

There is a need for an intelligent system capable of automated, real-time traffic analysis using AI and deep learning.

1.2 Motivation

The motivation of this research is to develop an automated traffic monitoring system that:

- Reduces human effort
- Provides real-time traffic insights

- Improves congestion management
- Enhances road safety
- Supports smart city infrastructure

1.3 Key objectives of this research include

The key objectives of this research are to design an AI-based traffic monitoring system using YOLOv4 for real-time vehicle detection and Deep SORT for accurate multi-object tracking; to analyze traffic density, vehicle count, and movement patterns using video data; to develop a scalable system capable of processing live and recorded video streams; to improve traffic management through data-driven insights; and to provide an efficient, cost-effective alternative to traditional traffic monitoring systems.

2. LITERATURE SURVEY

Recent advancements in computer vision and deep learning have significantly transformed intelligent transportation systems by enabling automated traffic monitoring, vehicle detection, and congestion analysis. Traditional traffic analysis methods relied on manual counting and sensor-based systems, which are inefficient and prone to errors. Modern approaches leverage deep learning models such as YOLO (You Only Look Once) for real-time object detection and Deep SORT for multi-object tracking. These methods improve accuracy, scalability, and real-time performance in traffic surveillance applications. The following table summarizes key contributions from existing research works relevant to the proposed system.

S.No	Citation	Research Focus	Methodology	Key Findings
1	Kusumah et al., 2023	Vehicle detection & counting	YOLOv4 + Deep SORT	Achieved efficient vehicle counting with tracking accuracy improvements
2	Zhang et al., 2022	Vehicle detection	Improved YOLOv5	Reduced false detection using Flip-Mosaic augmentation
3	Lin et al., 2021	Traffic monitoring	YOLO + GMM	Improved vehicle counting and speed estimation accuracy
4	Azimjonov et al., 2021	Vehicle detection & tracking	YOLO + BBox tracking	Increased detection accuracy up to 95% using hybrid models
5	Shubho et al., 2022	Traffic violation detection	YOLOv4 + OpenCV	Enabled real-time detection of traffic violations
6	Luo et al., 2024	Traffic monitoring	YOLOv5 + Deep SORT	Improved tracking stability and speed estimation
7	Vohra et al., 2023	Traffic monitoring	YOLO-based detection	Demonstrated efficiency of YOLO in real-time vehicle detection
8	Kunekar et al., 2024	Traffic signal optimization	YOLO + ML	Improved traffic flow using density-based signal control
9	Mandal et al., 2020	Vehicle counting	YOLO + Deep SORT	Achieved high accuracy in vehicle counting under varying conditions
10	Rezaei et al., 2021	Intelligent traffic systems	YOLO + SORT	Enabled 3D traffic monitoring and congestion detection

3. BACKGROUND WORK

The development of intelligent traffic monitoring systems has evolved significantly with advancements in computer vision, deep learning, and real-time analytics. Traditional traffic systems relied on manual observation and sensor-based infrastructure, which are limited in scalability and accuracy. The introduction of deep learning-based object detection and tracking models has enabled automated, accurate, and real-time traffic analysis.

3.1 Object Detection using YOLOv4

YOLOv4 (You Only Look Once version 4) is a real-time object detection model known for its speed and accuracy. Unlike traditional region-based detectors, YOLO processes the entire image in a single forward pass, making it highly efficient for real-time applications.

Key advantages:

- High detection speed (real-time performance)
- Ability to detect multiple objects simultaneously
- Robust performance under varying lighting conditions

In traffic analysis, YOLOv4 is used to detect different vehicle classes such as cars, buses, trucks, and motorcycles.

3.2 Multi-Object Tracking using Deep SORT

Deep SORT (Simple Online and Realtime Tracking with Deep Learning) extends the SORT algorithm by incorporating appearance features for improved tracking.

Key features:

- Assigns unique IDs to each vehicle
- Maintains identity across frames
- Prevents duplicate counting

This ensures accurate vehicle tracking even in crowded traffic scenarios.

3.3 Video Processing and Computer Vision Techniques

The system utilizes OpenCV and NumPy for handling video streams and preprocessing operations:

- Frame extraction from video
- Image resizing and normalization
- Noise reduction
- Bounding box refinement

These preprocessing steps improve detection accuracy and system performance.

3.4 Traffic Analytics and Pattern Recognition

Traffic analytics involves extracting meaningful insights from vehicle movement data, including:

- Vehicle density estimation
- Traffic flow patterns
- Congestion detection
- Peak-hour analysis

These analytics support intelligent traffic management and decision-making.

3.5 Intelligent Transportation Systems (ITS)

The integration of AI-based traffic analysis into ITS enables:

- Smart traffic signal control
- Automated congestion monitoring
- Real-time traffic alerts
- Data-driven urban planning

4. PROPOSED MODEL

The proposed system is an intelligent real-time road traffic analysis framework designed to automatically detect, track, and analyze vehicles using deep learning techniques. It integrates YOLOv4 for object detection and Deep SORT for multi-object tracking to ensure accurate identification and monitoring of vehicles across video frames. The system processes live or recorded video feeds and extracts meaningful insights such as traffic density and congestion patterns. The modular design ensures scalability, real-time performance, and adaptability for smart city applications.

4.1 Input Acquisition

The input acquisition module is responsible for capturing traffic video data either from live surveillance cameras or pre-recorded video files. It supports multiple video formats and ensures continuous frame streaming for processing. This module acts as the entry point of the system, providing raw visual data for further analysis. The quality and resolution of input data significantly influence detection accuracy and system performance.

4.2 Preprocessing

The pre-processing module prepares the input video frames for efficient analysis by applying several image processing techniques. It includes frame extraction, resizing, normalization, and noise reduction to improve image quality. These steps ensure consistency in input data and enhance the performance of the detection model. Proper preprocessing reduces computational complexity and increases the robustness of the system under varying lighting and environmental conditions.

4.3 Vehicle Detection (YOLOv4)

The vehicle detection module uses the YOLOv4 deep learning model to identify different types of vehicles in each frame. It generates bounding boxes around detected objects along with confidence scores indicating prediction accuracy. YOLOv4 processes images in a single pass, enabling real-time detection with high speed and precision. This module forms the core of the system by accurately identifying vehicles such as cars, buses, trucks, and motorcycles.

4.4 Vehicle Tracking (Deep SORT)

The tracking module utilizes the Deep SORT algorithm to monitor vehicles across consecutive frames. It assigns a unique ID to each detected vehicle and maintains its identity throughout the video sequence. By combining motion and appearance features, Deep SORT ensures accurate tracking even in crowded scenarios. This module prevents duplicate counting and enables analysis of vehicle movement patterns.

4.5 Traffic Analysis Engine

The traffic analysis module processes detection and tracking data to generate meaningful insights. It calculates vehicle counts, estimates traffic density, and identifies congestion zones in real time. The module also analyzes movement patterns and detects peak traffic conditions. These insights help in understanding traffic behavior and support intelligent decision-making for traffic management.

4.6 Data Management

The data management module is responsible for storing and organizing processed data, including video frames, detection results, and tracking information. It ensures efficient data handling and retrieval for analysis and reporting. This module also maintains historical records, enabling trend analysis and performance evaluation over time. Proper data management enhances system scalability and reliability.

4.7 Visualization Dashboard

The visualization module presents the output in an interactive and user-friendly format. It displays bounding boxes around detected vehicles, shows real-time vehicle counts, and provides graphical insights such as traffic density and congestion levels. The dashboard helps users easily interpret results and supports monitoring of traffic conditions. It enhances user experience and enables effective decision-making.

4.8 Parallel Processing

The system supports parallel processing to handle multiple tasks simultaneously, such as detection, tracking, and analysis. This ensures real-time performance even when processing high-resolution video streams. Parallel execution improves efficiency and reduces latency, making the system suitable for real-world deployment in busy traffic environments.

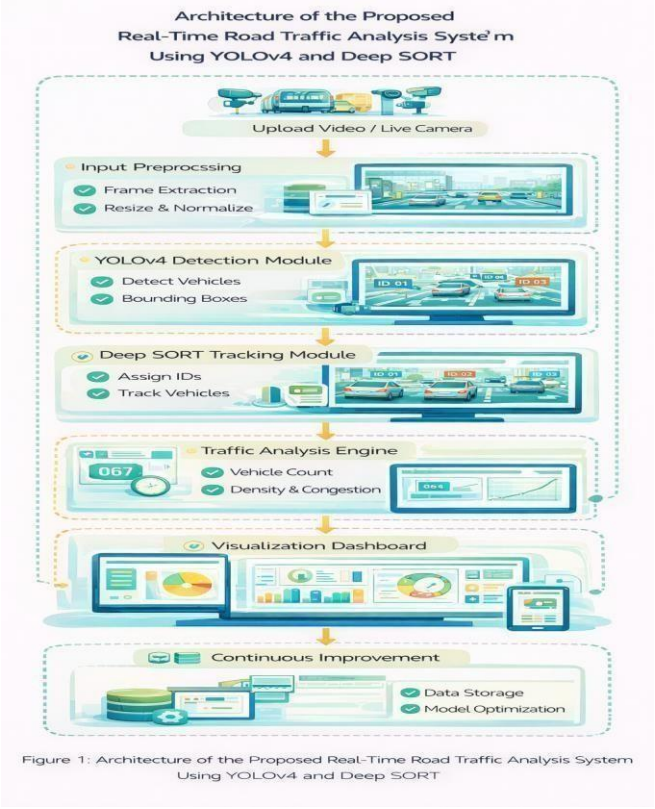

Figure 1: Architecture of the Proposed Real-Time Road Traffic Analysis System Using YOLOv4 and Deep SORT

Fig 1: Architecture of the Proposed Real-Time Road Traffic Analysis System Using YOLOv4 and Deep SORT

Fig 1 illustrates the overall architecture of the proposed real-time road traffic analysis system designed for intelligent vehicle detection, tracking, and congestion monitoring. The system begins with video input acquisition, where traffic data is captured through live surveillance cameras or uploaded video streams. The input is then processed

in the preprocessing stage, which includes frame extraction, resizing, and normalization to enhance image quality and ensure consistency.
The processed frames are passed to the YOLOv4 detection module, where vehicles such as cars, buses, and trucks are identified using bounding boxes and confidence scores. Following detection, the Deep SORT tracking module assigns unique IDs to each vehicle and tracks their movement across frames, ensuring accurate counting and preventing duplication.

5. IMPLEMENTATION RESULTS

The XOJ system was implemented as a complete web-based application integrating machine learning, explainable AI, and visualization tools.

System Implementation Overview
The implementation includes:

- Backend development using Flask
- Frontend design using modern UI frameworks
- Integration of ML models and XAI tools
  The modular structure ensures scalability and maintainability.

1) Front Page:

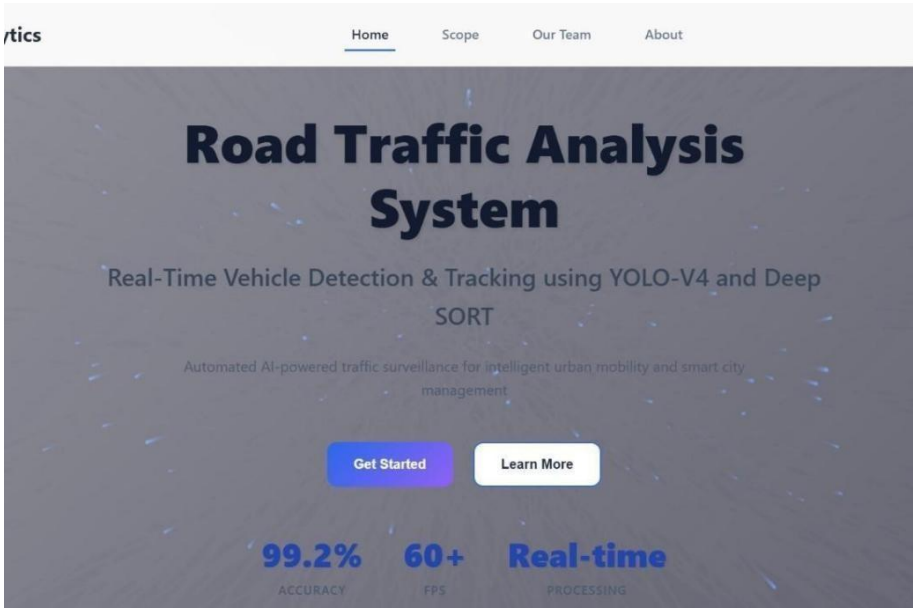

Fig 2:Introduction Page.

Fig 2 explains about the initial access point of the system that allows authorized users to securely enter the application. This interface typically contains input fields for the username and password along with a login button to authenticate the user. It ensures that only valid users can access the emotion detection system and its features.

2) Student Analyzer

Fig 3. XOJ Student Analyzer Interface

Fig 3 explains the interface where the system takes user input and try to analyze the student performance.

3) Results Page

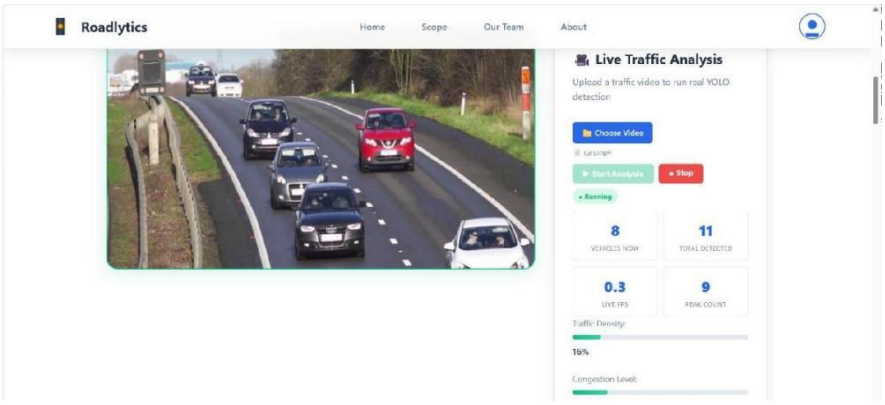

Fig 4. Analyze Student Performance without creating an account

Fig 4 displays the system displays feature contribution visualizations to explain the prediction outcome.

CONCLUSION

The proposed Student Performance Prediction System for Online Judge platforms effectively demonstrate the integration of machine learning and explainable artificial intelligence to enhance academic evaluation processes. By incorporating behavioral learning analytics such as submission patterns, consistency, and engagement levels, the system moves beyond traditional score-based assessment and provides a more comprehensive understanding of student performance. The implementation of the Random Forest algorithm ensures reliable and accurate predictions, while the integration of SHAP-based explainable AI enhances transparency by clearly identifying the contribution of each feature to the final prediction. This interpretability enables educators to gain deeper insights into student learning behavior and supports informed decision-making for early intervention. The system's web-based architecture, built using Flask and SQLite, ensures scalability, usability, and efficient data management. Visualization tools such as dashboards, confusion matrices, and performance graphs further improve user interaction and understanding of analytical results. Overall, the proposed framework provides a robust, scalable, and interpretable solution for predicting student performance in online judge environments. It contributes significantly to the advancement of educational data mining and intelligent learning systems. Future enhancements can focus on real-time analytics, integration with learning management systems, and the use of advanced deep learning models to further improve prediction accuracy and system adaptability.

REFERENCES

1. A. P. Kusumah, D. Djayusman, G. R. Setiadi, A. C. Nugraha, and P. Hidayatullah, "Counting Various Vehicles using YOLOv4 and DeepSORT," *Journal of Integrated and Advanced Engineering*, vol. 3, no. 1, pp. 1–6, 2023.
2. Y. Zhang, Z. Guo, J. Wu, Y. Tian, H. Tang, and X. Guo, "Real-Time Vehicle Detection Based on Improved YOLOv5," *Sustainability*, vol. 14, no. 19, 2022.
3. C.-J. Lin, "A Real-Time Vehicle Counting, Speed Estimation, and Classification System Based on YOLO," *Mathematical Problems in Engineering*, 2021.
4. J. Azimjonov et al., "A Real-Time Vehicle Detection and Tracking System Using YOLO," *Expert Systems with Applications*, 2021.
5. F. H. Shubho, F. Iftekhar, E. Hossain, and S. Siddique, "Real-Time Traffic Monitoring and Traffic Offense Detection Using YOLOv4," 2022.
6. Z. Luo et al., "Enhanced YOLOv5s and Deep SORT for Highway Vehicle Tracking," *Frontiers in Physics*, 2024.
7. D. S. Vohra et al., "Real-Time Vehicle Detection for Traffic Monitoring Using YOLO," *International Journal of Intelligent Unmanned Systems*, 2023.
8. P. Kunekar et al., "Traffic Management System Using YOLO Algorithm," *MDPI Proceedings*, 2024.
9. V. Mandal and Y. Adu-Gyamfi, "Object Detection and Tracking Algorithms for Vehicle Counting," *arXiv*, 2020.
10. M. Rezaei, M. Azarmi, and F. M. Pour Mir, "Traffic-Net: 3D Traffic Monitoring Using a Single Camera," *arXiv*, 2021.